



PYTHON GAMES (PYGAME)

Alexandre Altair de Melo

Disciplina: CGA - Prof. Dr. Marcelo da Silva Hounsell

22/06/2026

Agenda

2

- ❑ Objetivos;
- ❑ Desenvolvendo jogos;
- ❑ Plataforma T-TEA;
- ❑ Pygame;
- ❑ OpenCV e NumPy;
- ❑ MediaPipe;
- ❑ Juntando as partes;
- ❑ E agora?
- ❑ Referências.

Objetivos

3

- ▣ Apresentar uma visão geral do desenvolvimento de jogos usando Pygame e visão computacional.
- ▣ Fontes:
 - <https://github.com/peregrinodeti/treinamentopygame>
- ▣ Agenda 1 Hora de Reunião por equipe:
 - <https://calendly.com/alexandremelo-br/1-hora-reuniao>
- ▣ Vamos usar a versão on-line do python para alguns exemplos:
 - <https://onecompiler.com/pygame>
- ▣ Para imagens vamos ter de fazer uma adaptação do código:
 - `import urllib.request`
 - `url_imagem = "https://site/imagem"`
 - `urllib.request.urlretrieve(url_imagem, "x.jpg")`
 - `imagem = cv2.imread("x.jpg")`

Desenvolvendo jogos

4

- ▣ Desenvolver jogos envolve várias etapas. A seguir apresentaremos algumas ferramentas utilizadas na plataforma T-TEA.

Plataforma T-TEA

5

- Conforme Trindade, Pereira e Hounsell (2022) a plataforma T-TEA é um chão interativo que utiliza equipamentos convencionais como projetor de vídeo, câmera, computador e uma estrutura suporte opcional para projetar o JS no chão. Figura 1 – Plataforma.

Figura 1 – Plataforma T-TEA



Fonte: Trindade, Pereira e Hounsell (2022)(Figura 1).

Plataforma T-TEA

6

- Conforme Trindade, Pereira e Hounsell (2022) a plataforma pode ser assim caracterizada:
 - Do ponto de vista do hardware:
 - Um computador;
 - Uma webcam;
 - Um projetor.
 - Do ponto de vista do software bibliotecas:
 - Python;
 - Pygame;
 - OpenCV;
 - Mediapipe.
 - Obs: Detalhe para inclinação do projetor a 45 graus, fixado na altura de 1,80 m. Numa área de projeção de 5m².

Plataforma T-TEA

7

▣ **Jogo:** RepeTEA Figura 2 e Perspectiva Figura 3.

Figura 3 – Perspectiva

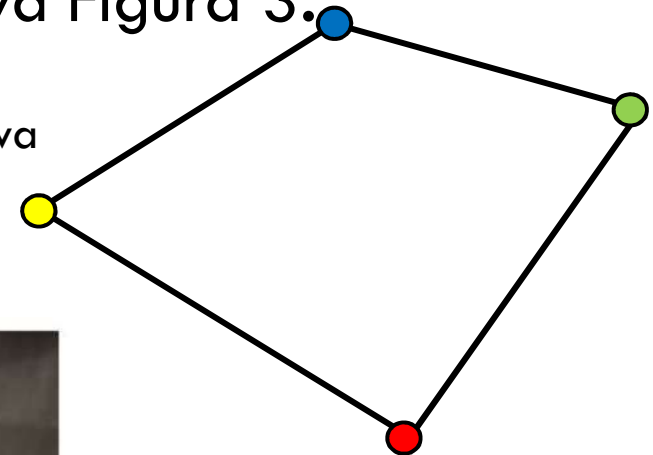
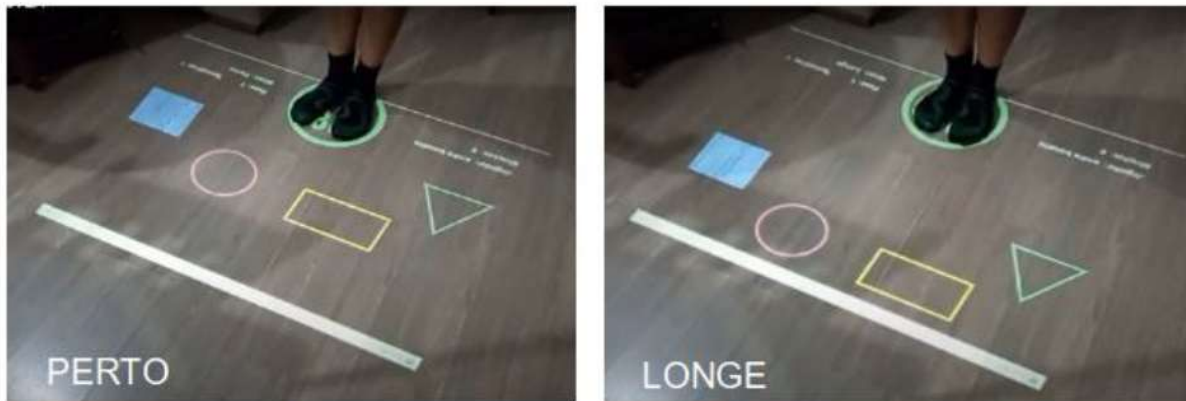


Figura 2 – Jogo RepeTEA



Fonte: Trindade, Pereira e Hounsell (2022)(Figura 3 (a)(b)).



Fonte: Elaborado pelo autor (2026).

Pygame

8

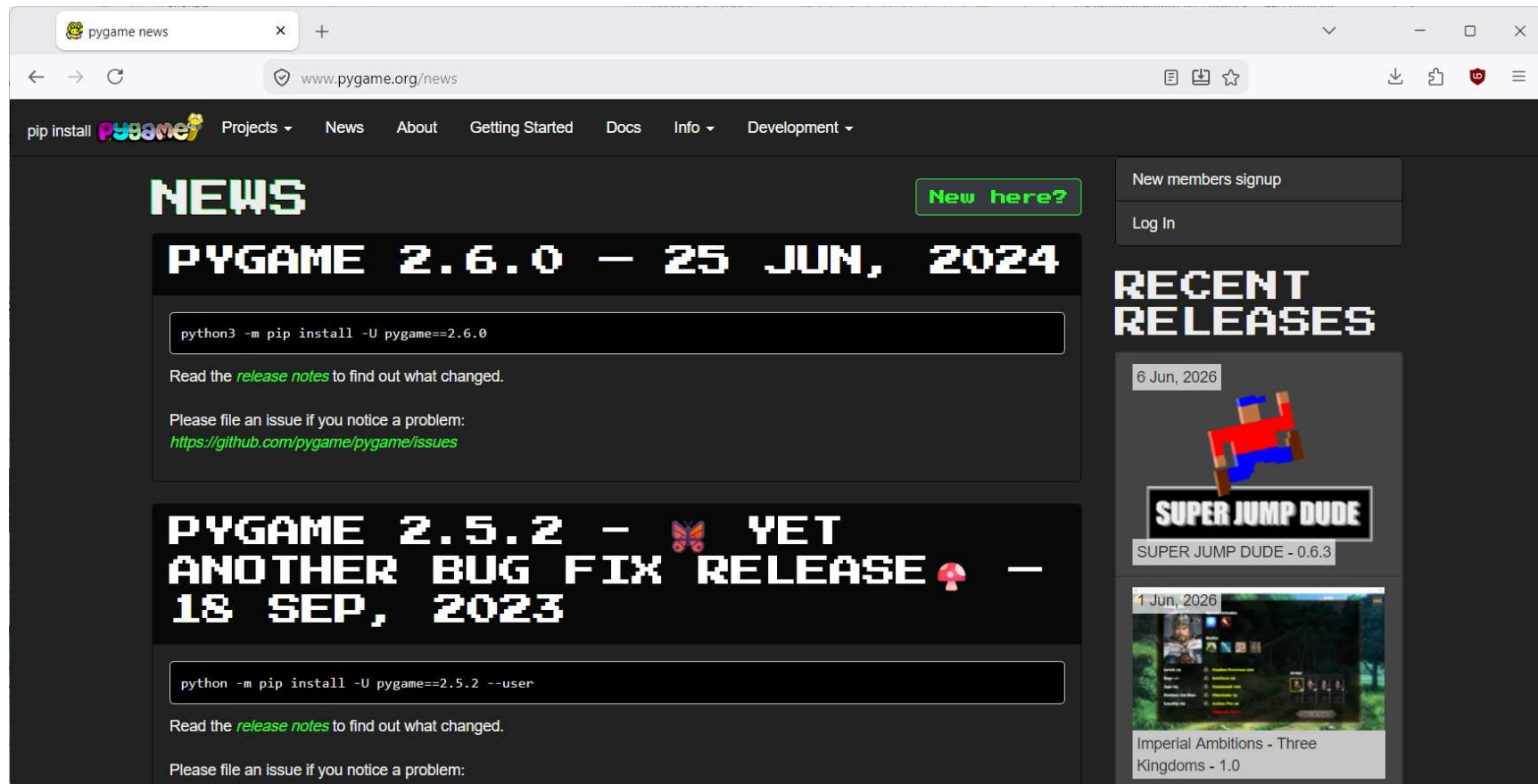
- ▣ **Sobre:** O Pygame é uma das bibliotecas de código aberto mais populares e tradicionais para o desenvolvimento de jogos e aplicações multimídia em Python. Desenvolvido sobre a biblioteca de baixo nível SDL (Simple DirectMedia Layer), ele permite o gerenciamento de gráficos, sons, eventos de **teclado/mouse** e física básica de forma muito direta;
- ▣ **Para que tipo de jogo é recomendada:** Jogos 2D Retro e Pixel Art. Vantagem de prototipagem rápida;
- ▣ **Versão atual:** 2.6.0;
- ▣ **Lançamento:** 06/2024;
- ▣ **Compatível:** Linux, Mac e Windows.

Pygame

9

▣ **Site:** www.pygame.org. Figura 4.

Figura 4 – Site PyGame



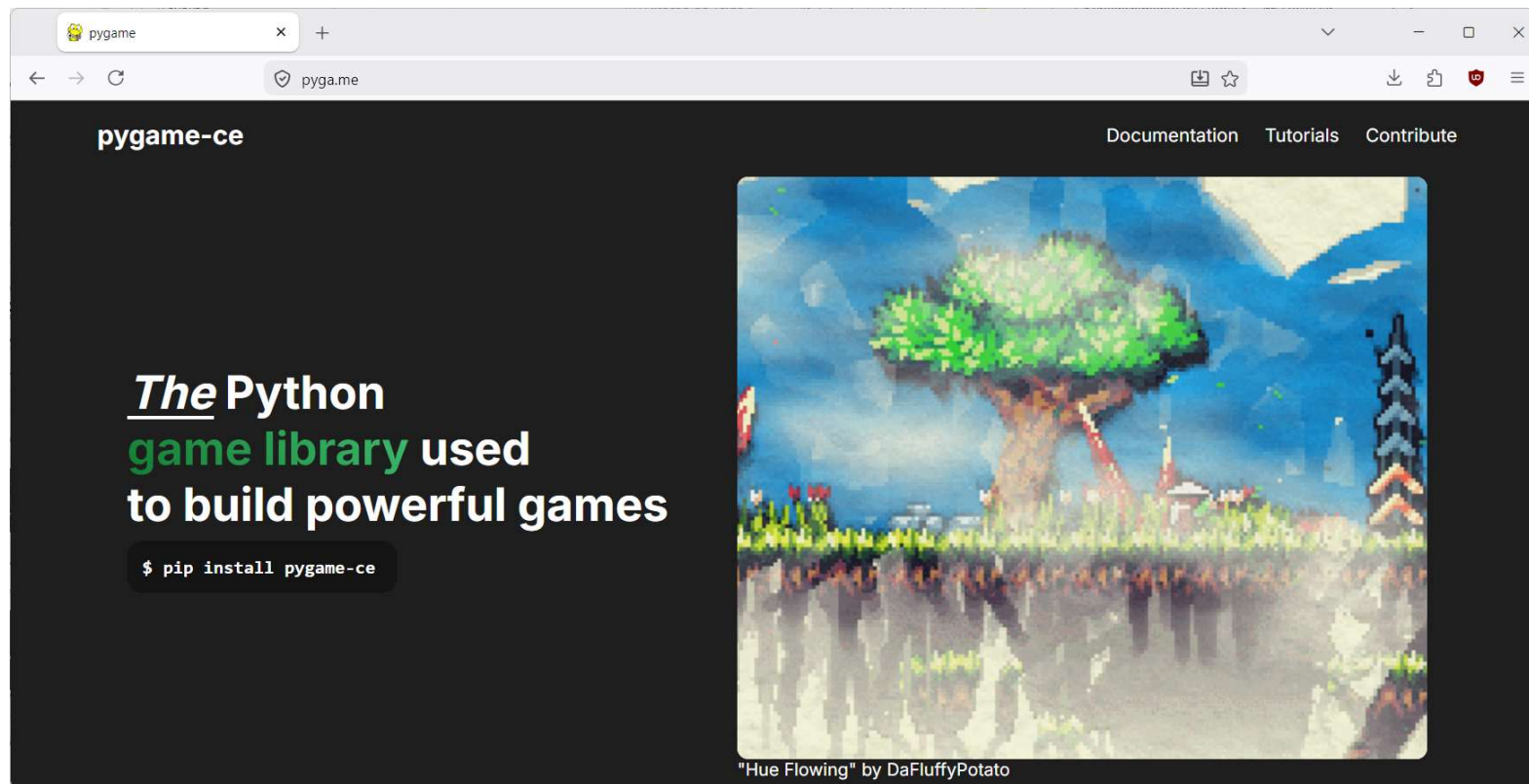
Fonte: Pygame (2026).

Pygame

10

- ▣ **Alternativa: Pygame Community Edition, versão 2.5.7 de 03/2026;**
- ▣ **Site: <https://pyga.me/> - Figura 5.**

Figura 5 – Site PyGame - CE



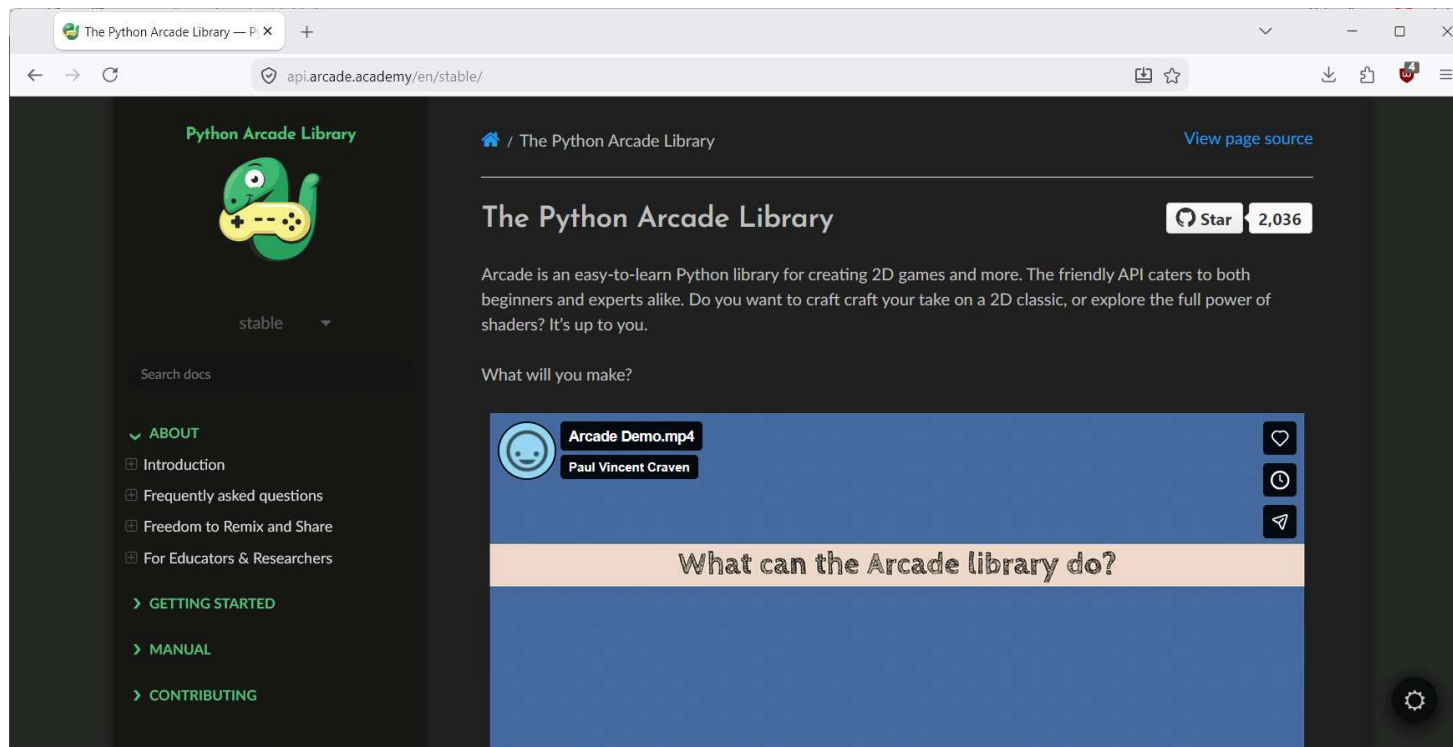
Fonte: PyGame - CE (2026).

Pygame

11

- **Alternativa: Python Arcade Library, versão 3.3.3 de 10/2025;**
- **Site: <https://api.arcade.academy/en/stable/> - Figura 6.**

Figura 6 – Site Python Arcade Library



Fonte: The Python Arcade Library (2026).

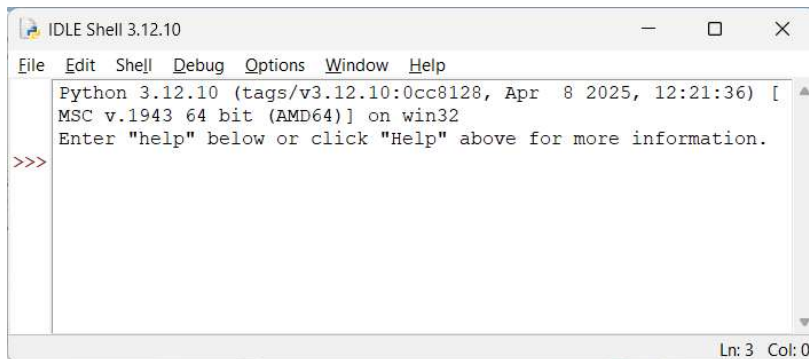
Pygame

12

▣ Ferramentas de desenvolvimento – Figura 7:

Figura 7 – Logo de Ferramentas

Gratuitas



Visual Studio Code

Pagas



*Sublime
Text*



PyCharm

Pygame

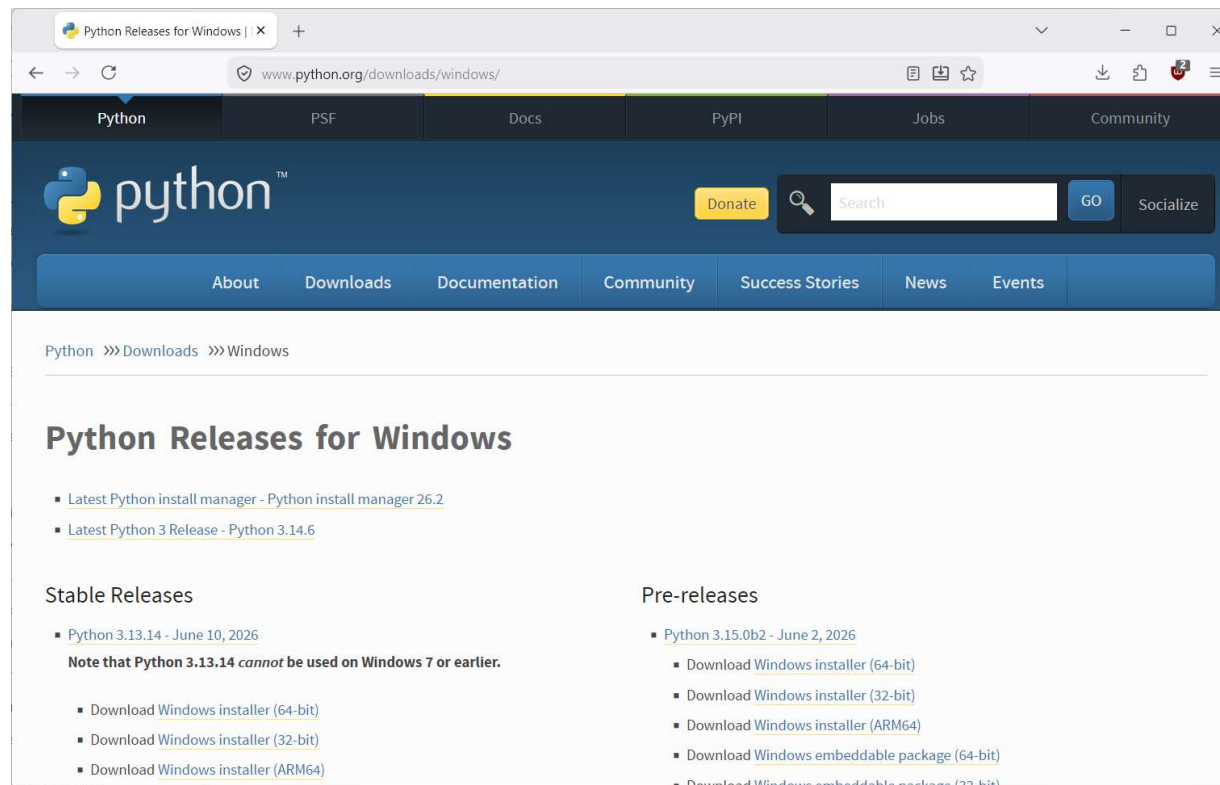
13

■ Instalação:

■ Para Windows do Python – Figura 8.

■ <https://www.python.org/downloads/windows/>

Figura 8 – Site Python



Fonte: Python (2026).

Pygame

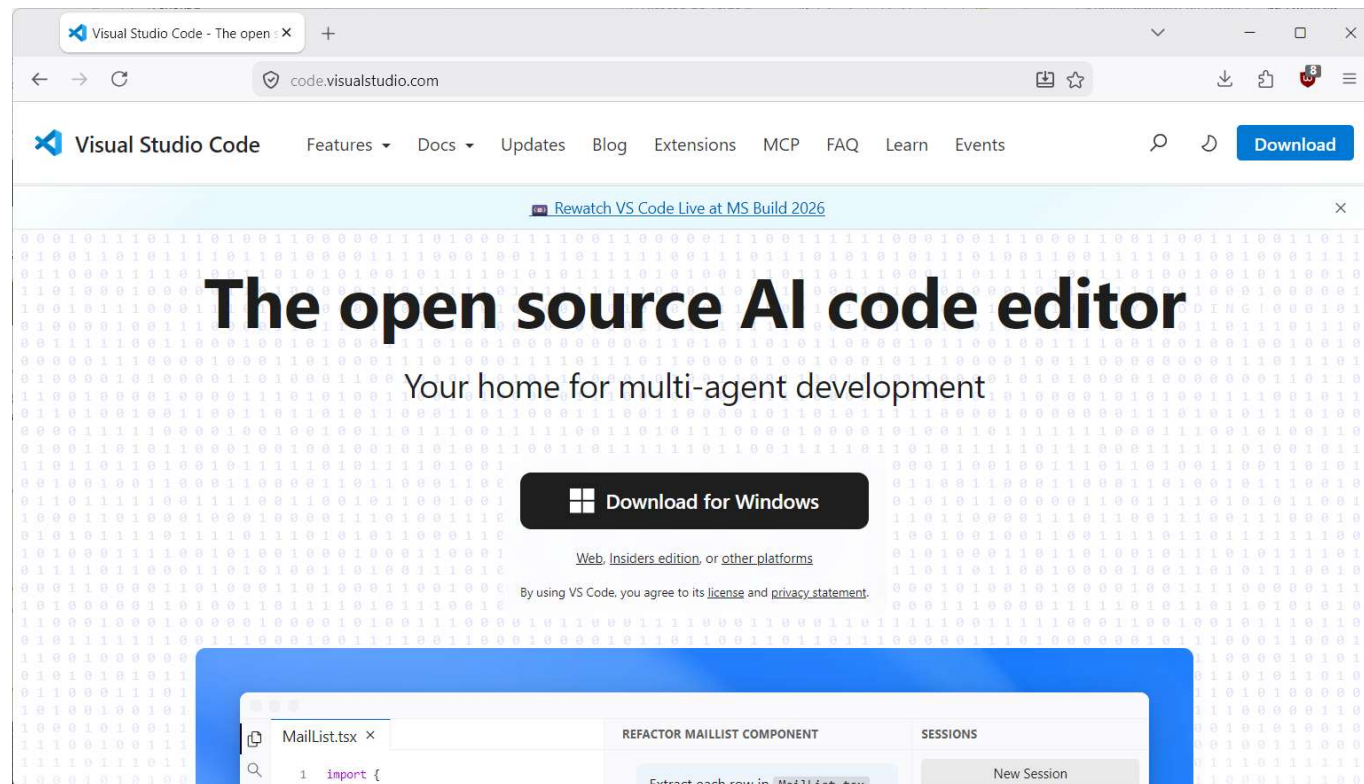
14

■ Instalação:

■ Para Windows Visual Studio Code – Figura 9.

■ <https://code.visualstudio.com/>

Figura 9 – Site VS Code



Fonte: Microsoft (2026).

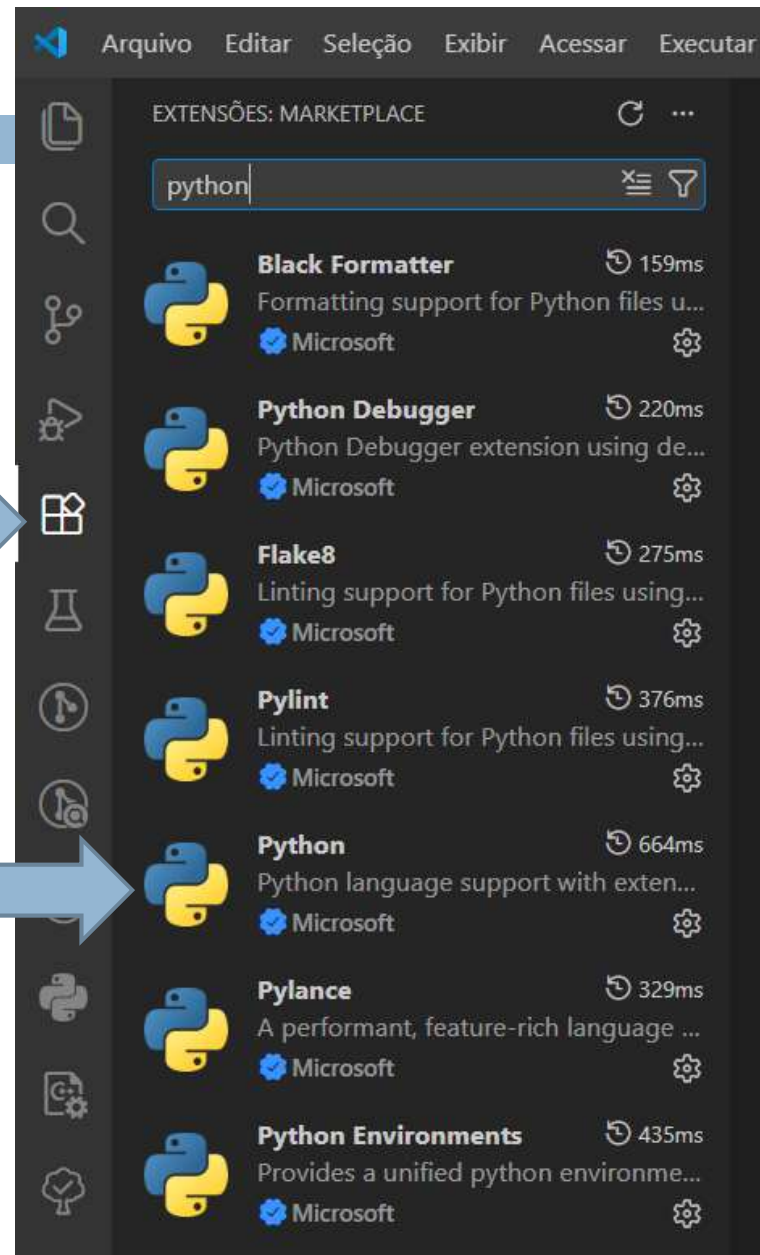
Pygame

15

■ Instalação:

- Complementos
- para Visual Studio Code
- Figura 10.

Figura 10 – Complementos VS



Fonte: Elaborado pelo autor (2026).

Pygame

16

▣ Instalação – Figura 11:

Figura 11 – Instalando PyGame



**GETTINGSTARTED —
WIKI**

PYGAME INSTALLATION

Pygame requires Python; if you don't already have it, you can download it from python.org. It's recommended to run the latest python version, because it's usually faster and has better features than the older ones. Bear in mind that pygame has dropped support for python 2.

The best way to install pygame is with the [pip](https://pip.pypa.io) tool (which is what python uses to install packages). Note, this comes with python in recent versions. We use the --user flag to tell it to install into the home directory, rather than globally.

```
python3 -m pip install -U pygame --user
```

INSTALAR

Fonte: Pygame (2026).

Abra um prompt de comando (terminal) no VS e digite: ***pip show pygame***

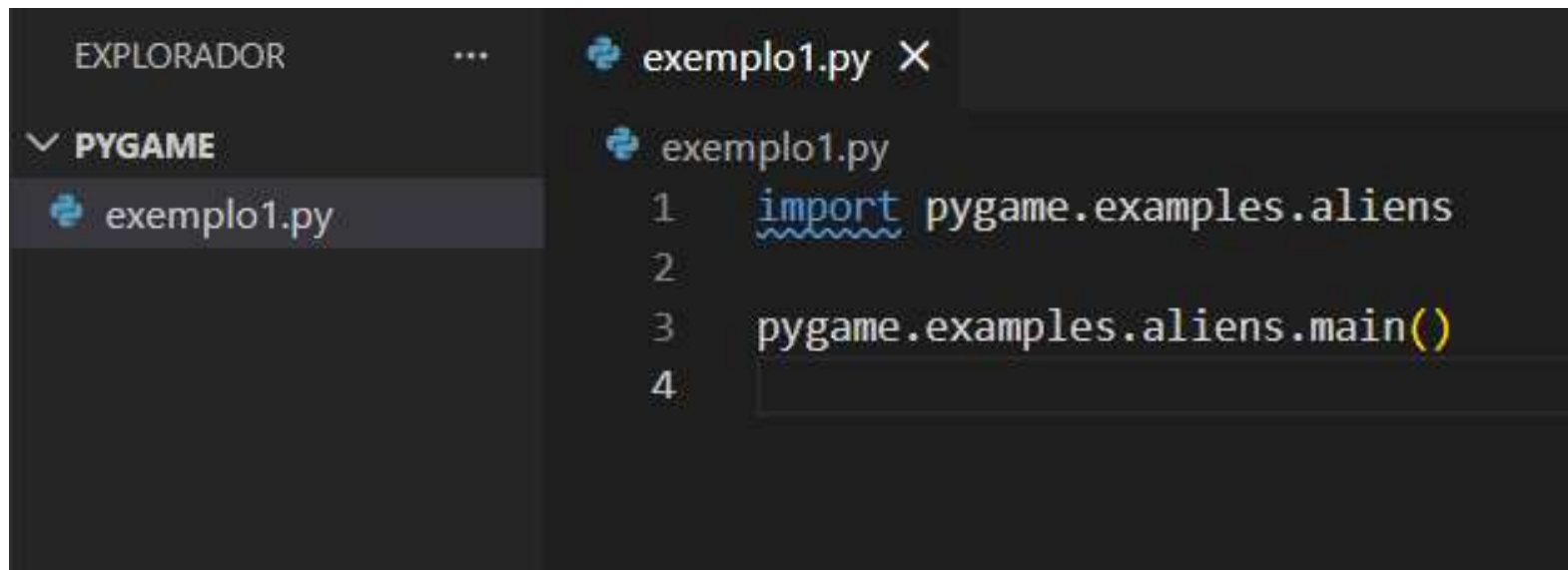
Pygame

17

■ Mais exemplos:

- Abra a pasta aonde vai ficar o seu código, por exemplo, **pygame**;
- Crie um arquivo **.py** no VS, por exemplo, **exemplo1.py**;
- Digite no **exemplo1.py** o seguinte código – Figura 1 2:

Figura 1 2 – Exemplos PyGame



The screenshot shows a code editor interface with a dark theme. On the left, the 'EXPLORADOR' (Explorer) sidebar is visible, showing a folder named 'PYGAME' which contains a file named 'exemplo1.py'. The main editor area displays the contents of 'exemplo1.py', which includes the following code:

```
1 import pygame.examples.aliens
2
3 pygame.examples.aliens.main()
4
```

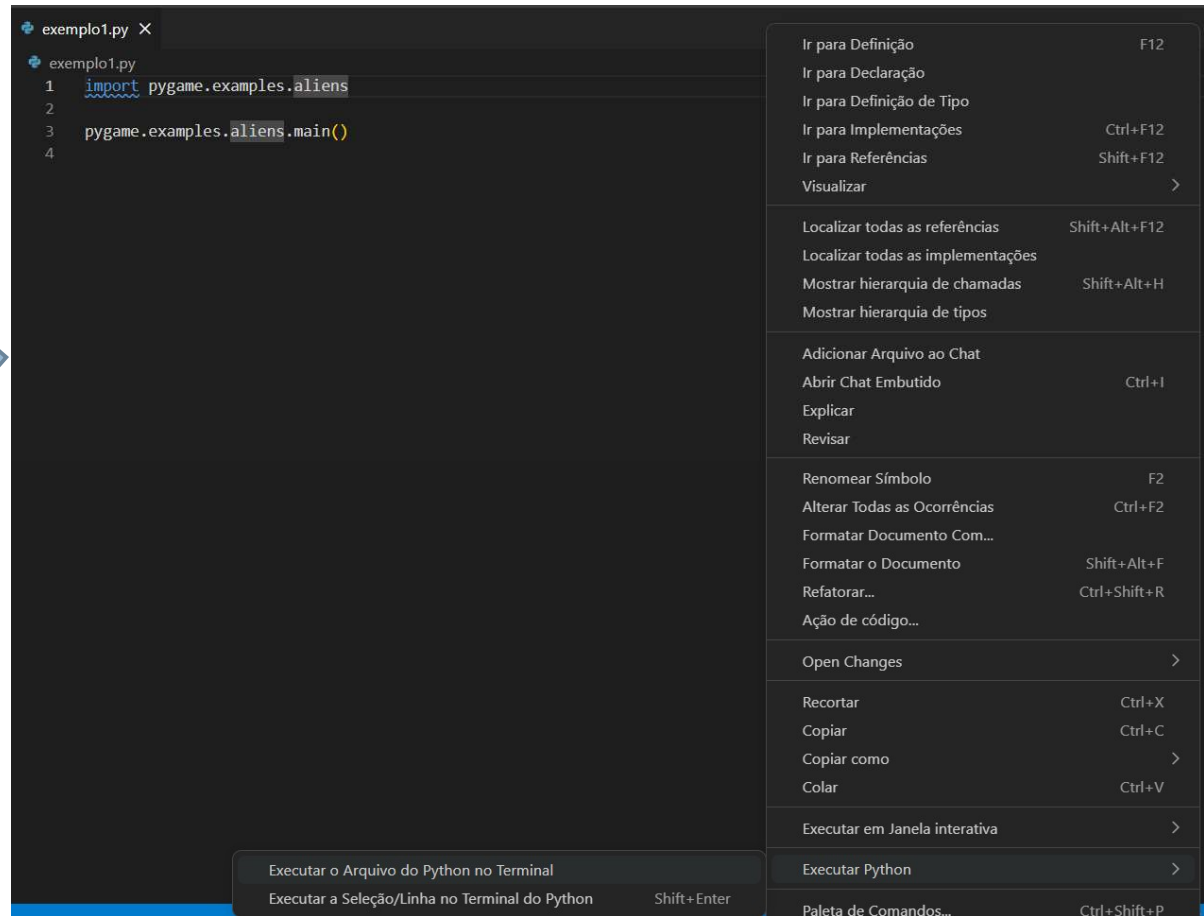
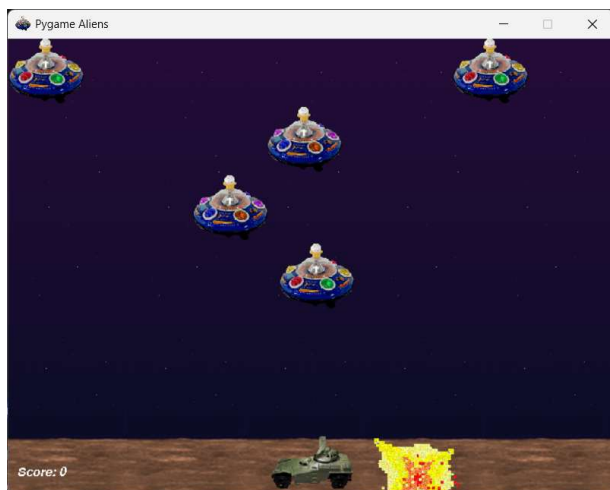
Pygame

18

▣ Para executar

- Botão direito do mouse e
- acesse a opção para
- execução, atenção
- ao caminho de execução
- no terminal – Figura 13.

Figura 13 – PyGame em Execução



Fonte: Elaborado pelo autor (2026).

Pygame

19

▣ Mais exemplos site do pygame -> docs – Figura 14:

Figura 14 – Documentos PyGame



Fonte: Pygame (2026).

| examples |

Pygame

20

▣ Explorando o loop básico 1/2:

- Crie o arquivo **exemplo2.py**, desafio troque o fundo para verde!

```
# Example file showing a basic pygame "game loop"
import pygame

# pygame setup
pygame.init()
screen = pygame.display.set_mode((1280, 720))
clock = pygame.time.Clock()
running = True

while running:
    # poll for events
    # pygame.QUIT event means the user clicked X to close your window
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # fill the screen with a color to wipe away anything from last frame
    screen.fill("purple")

    # RENDER YOUR GAME HERE

    # flip() the display to put your work on screen
    pygame.display.flip()

    clock.tick(60) # Limits FPS to 60

pygame.quit()
```

Figura 15 – Exemplo Fundo Verde

Exemplo disponível em:
<https://www.pygame.org/docs/>

Pygame

21

■ Explorando o loop básico 2/2:

- Crie o arquivo **exemplo2.py**, desafio troque o fundo para verde!

```
# Example file showing a basic pygame "game Loop"
import pygame

# pygame setup
pygame.init()
screen = pygame.display.set_mode((1280, 720))
clock = pygame.time.Clock()
running = True

while running:
    # poll for events
    # pygame.QUIT event means the user clicked X to close your window
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # fill the screen with a color to wipe away anything from last frame
    screen.fill("purple")

    # RENDER YOUR GAME HERE

    # flip() the display to put your work on screen
    pygame.display.flip()

    clock.tick(60) # Limits FPS to 60

pygame.quit()
```

Black
Blue
White
Yellow
Green
Red
Brown

Pygame

22

- **Cores personalizadas:** Pode ser feito via o padrão R(ed)G(reen)B(lue).
 - Roxo em RGB
 - `screen.fill("purple")`
 - `screen.fill((128, 0, 128))`
 - Com constante:
 - `PURPLE = (128, 0, 128)`
 - `screen.fill(PURPLE)`
 - Variação RGBA(lpha)
 - `screen.fill((128, 0, 128, 128))` # roxo semi-transparente

Pygame

23

■ Explorando o loop básico 1/2:

■ Crie o arquivo exemplo3.py.

```
# Example file showing a circle moving on screen
import pygame

# pygame setup
pygame.init()
screen = pygame.display.set_mode((1280, 720))
clock = pygame.time.Clock()
running = True
dt = 0

player_pos = pygame.Vector2(screen.get_width() / 2, screen.get_height() / 2)

while running:
    # poll for events
    # pygame.QUIT event means the user clicked X to close your window
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # fill the screen with a color to wipe away anything from last frame
    screen.fill("purple")

    pygame.draw.circle(screen, "red", player_pos, 40)

    keys = pygame.key.get_pressed()
    if keys[pygame.K_w]:
        player_pos.y -= 300 * dt
    if keys[pygame.K_s]:
        player_pos.y += 300 * dt
    if keys[pygame.K_a]:
        player_pos.x -= 300 * dt
    if keys[pygame.K_d]:
        player_pos.x += 300 * dt

    # flip() the display to put your work on screen
    pygame.display.flip()

    # Limits FPS to 60
    # dt is delta time in seconds since last frame, used for framerate-
    # independent physics.
    dt = clock.tick(60) / 1000

pygame.quit()
```

Fonte: Adaptado de PyGame (2026).

Figura 16 – Exemplo Eventos

- Desafio:
 - 1 – Troque a cor da bola para azul;
 - 2 – Troque o fundo para branco;
 - 3 – Mude a velocidade de movimentação da bola para 10x mais rápido e 10x mais lento;
 - 4 – Mude os controles de direção para as setas.

Exemplo disponível em:
<https://www.pygame.org/docs/>

Pygame

24

■ Explorando o loop básico 2/2:

■ Crie o arquivo exemplo3.py.

```
# Example file showing a circle moving on screen
import pygame

# pygame setup
pygame.init()
screen = pygame.display.set_mode((1280, 720))
clock = pygame.time.Clock()
running = True
dt = 0

player_pos = pygame.Vector2(screen.get_width() / 2, screen.get_height() / 2)

while running:
    # poll for events
    # pygame.QUIT event means the user clicked X to close your window
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # fill the screen with a color to wipe away anything from last frame
    screen.fill("purple")

    pygame.draw.circle(screen, "red", player_pos, 40)

    keys = pygame.key.get_pressed()
    if keys[pygame.K_w]:
        player_pos.y -= 300 * dt
    if keys[pygame.K_s]:
        player_pos.y += 300 * dt
    if keys[pygame.K_a]:
        player_pos.x -= 300 * dt
    if keys[pygame.K_d]:
        player_pos.x += 300 * dt

    # flip() the display to put your work on screen
    pygame.display.flip()

    # Limits FPS to 60
    # dt is delta time in seconds since last frame, used for framerate-
    # independent physics.
    dt = clock.tick(60) / 1000

pygame.quit()
```

Black, Blue, White, Yellow,
Green, Red, Brown...

Up, Down, Left e Right

Tempo...

Pygame

25

▣ Módulos Básicos – Figura 17

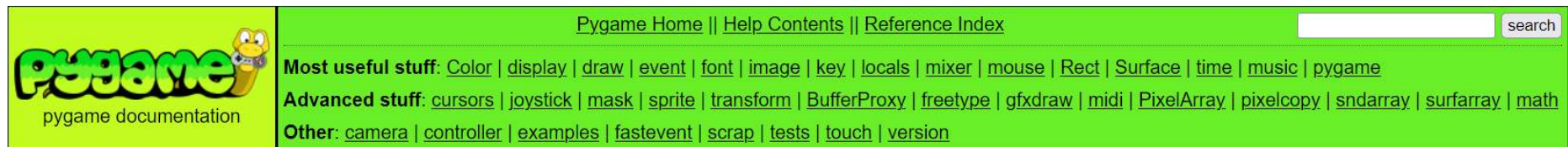
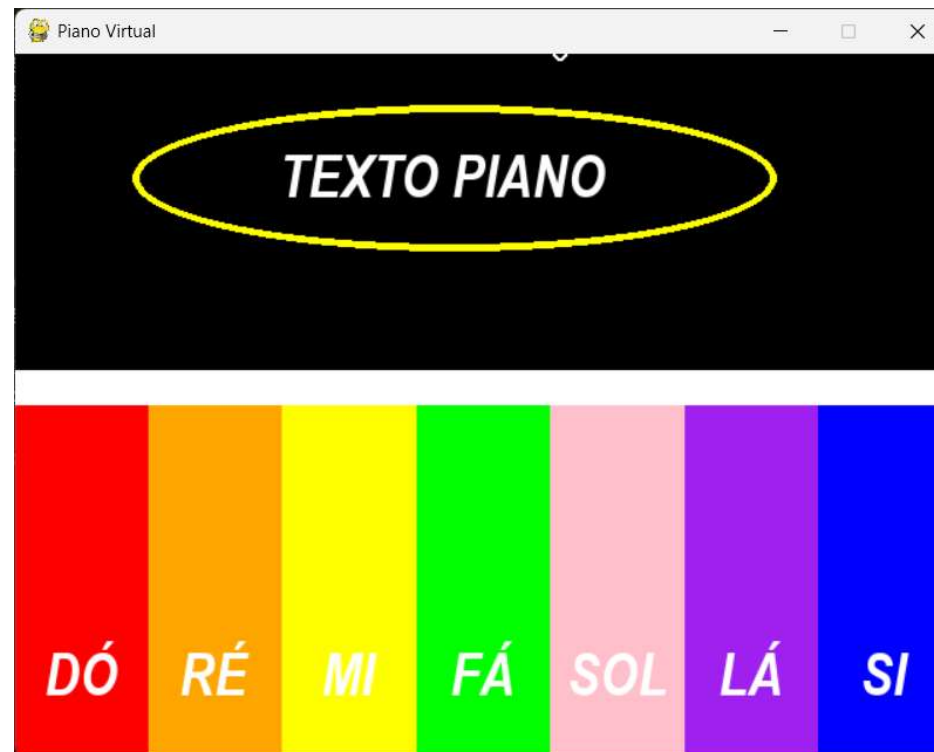


Figura 17 – PyGame Piano



Fonte: Elaborado pelo autor (2026).

Pygame

26

- Construindo uma tela, capturando um evento e saindo.
Exemplo: **piano.py**. – Figura 18.

Figura 18 – PyGame Piano Código

```
import pygame

pygame.init()
# Constantes para a tela
LARGURA_TELA = 640
ALTURA_TELA = 480
tela = pygame.display.set_mode((LARGURA_TELA, ALTURA_TELA))
# Título da janela
pygame.display.set_caption("Piano Virtual")

clock = pygame.time.Clock()
running = True

# Inicia o loop principal do jogo
while running:
    # Nesse ponto dentro desse laço é colocado o jogo para rodar,
    # ou seja, o que deve acontecer a cada frame

    # Observa os eventos de teclado e mouse
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Limpa a tela com uma cor de fundo antes de desenhar
    tela.fill("black")

    # Atualiza a tela com tudo o que foi desenhado acima
    pygame.display.flip()

    # Controla a taxa de quadros (FPS)
    clock.tick(60)

pygame.quit()
```

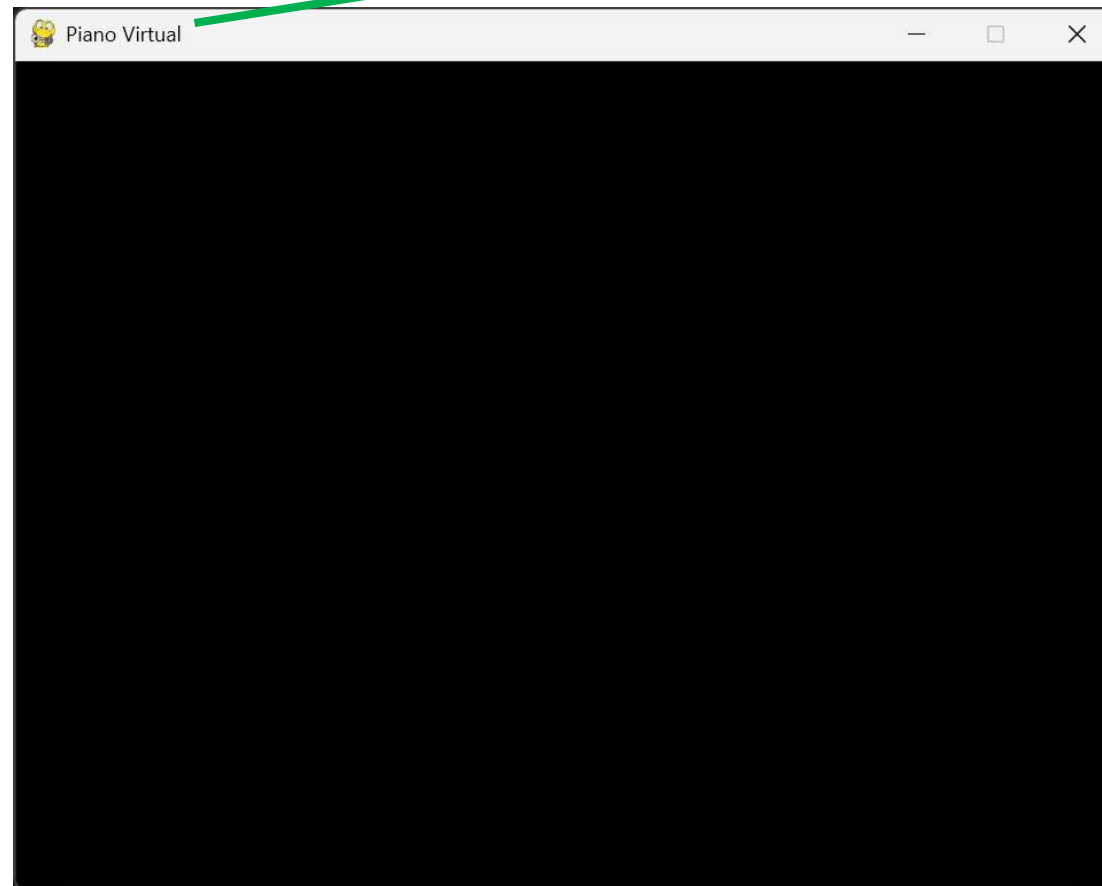
Fonte: Elaborado pelo
autor (2026).

Pygame

27

▣ Resultado de **piano.py**.

Título da Janela



Altura 480px

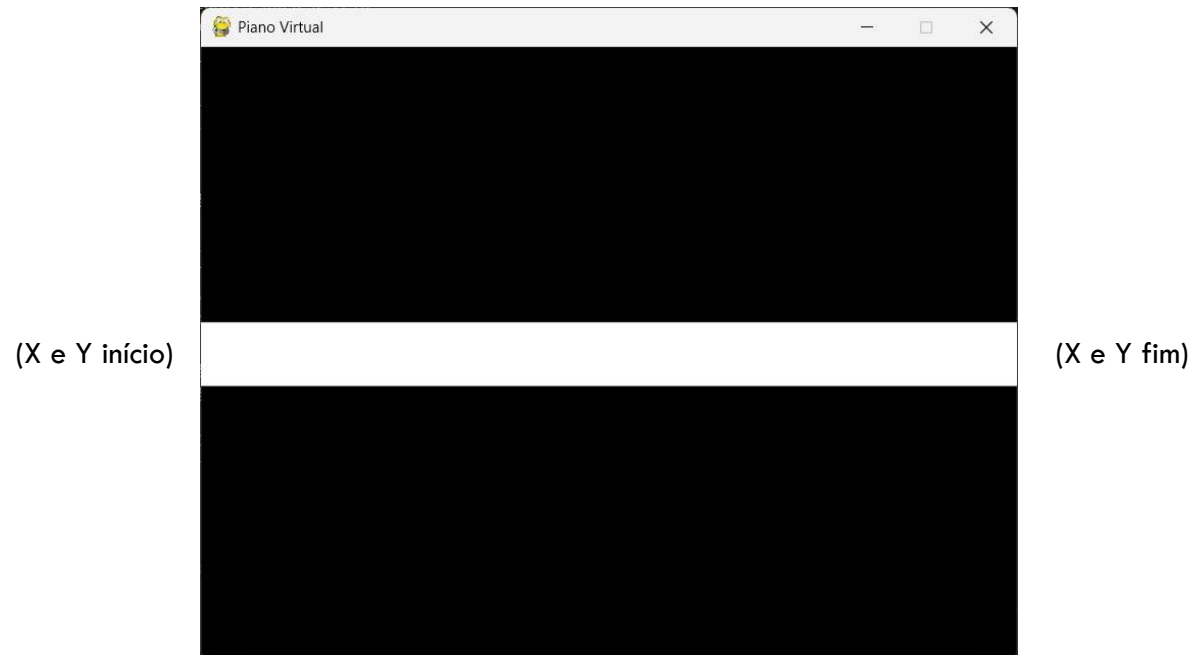
Largura 640px

Pygame

28

- ▣ **Linha:** Crie um novo arquivo chamado **piano_linha.py** com base no **piano.py**. E coloque esse trecho entre a linha **tela.fill("black")** e **pygame.display.flip()**.

```
pygame.draw.line(tela, "white", (0, 240), (640, 240), 50)
```



Pygame

29

- ▣ **Retângulo:** Crie um novo arquivo chamado **piano_retangulo.py** com base no **piano_linha.py**. E troque a linha **pygame.draw.line(...)**. Por esse código: - Figura 19

```
pygame.draw.rect(tela, "yellow", (0, 0, 640, 240))
```

(X e Y referência)

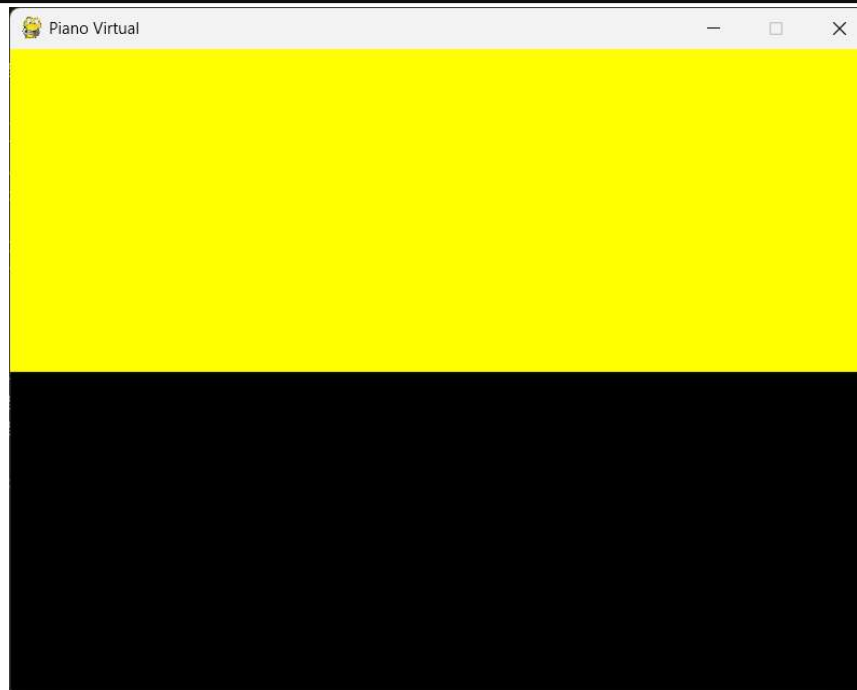


Figura 19 – PyGame
Piano Linha

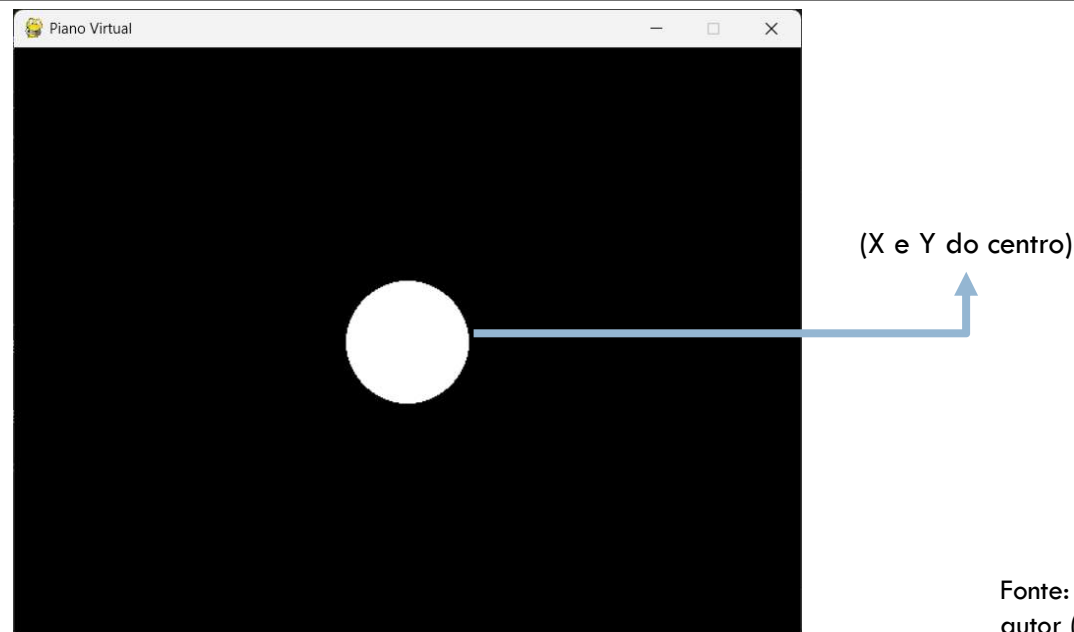
Pygame

30

- **Círculo:** Crie um novo arquivo chamado **piano_circulo.py** com base no **piano_linha.py**. E troque a linha **pygame.draw.line(...)**. Por esse código – Figura 20:

```
pygame.draw.circle(
    tela, "white", ((LARGURA_TELA / 2), (ALTURA_TELA / 2)), 50
)
```

Figura 20 – PyGame
Piano Círculo



Fonte: Elaborado pelo
autor (2026).

Pygame

31

- ▣ **Ellipse:** Crie um novo arquivo chamado **piano_ellipse.py** com base no **piano_linha.py**. E troque a linha **pygame.draw.line(...)**. Por esse código – Figura 21:

```
pygame.draw.ellipse(tela, "yellow", (80, 35, 440, 100), 5)
```

Figura 21 – PyGame
Piano Elipse



Fonte: Elaborado pelo
autor (2026).

Pygame

32

- ▣ **Fonte:** Crie um novo arquivo chamado **piano_fonte.py** com base no **piano_linha.py**. E troque a linha **pygame.draw.line(...)**. Por esse código – Figura 22:

```
fonte = pygame.font.SysFont("Arial", 40, bold=True, italic=True)
texto = fonte.render("TEXTO PIANO", True, "white")
tela.blit(texto, [(180), (60)])
```

Pega uma fonte do sistema

Adiciona o texto renderizado na superfície escolhida



Renderiza o texto com a fonte escolhida

Figura 22 – PyGame Piano Fonte

Fonte: Elaborado pelo autor (2026).

Pygame

33

- **Sons:** Crie um novo arquivo chamado **piano_som.py** com base no **piano_linha.py**. E troque a linha **pygame.draw.line(...)**. Por esse código – Figura 23:

```
som_do = pygame.mixer.Sound("sons/do.mp3")  
som_do.play()
```

Chama o mixer
da biblioteca
e define o
caminho do
som

Toca o som



Figura 23 – PyGame
Piano Som

Fonte: Elaborado pelo
autor (2026).

Pygame

34

- **Mouse Evento:** Crie um novo arquivo chamado **posicao_mouse.py** com base no **piano_linha.py**. E troque a linha **pygame.draw.line(...)**. Por esse código - Figura 24:

```
posicao_mouse = pygame.mouse.get_pos()
print(f"Posição do mouse: {posicao_mouse}")
```

Figura 24 – PyGame
Piano Mouse

```
Posição do mouse: (518, 479)
Posição do mouse: (518, 479)
Posição do mouse: (518, 479)
```

A posição X,Y do
mouse no terminal

Pygame

35

- **Mouse Evento:** Crie um novo arquivo chamado **evento_mouse.py** com base no **piano.py**. E refaça a parte de eventos adicionando esses dois a mais. Por esse código – Figura 25:

Figura 25 – PyGame
Piano Mouse Evento

```
# Observa os eventos de teclado e mouse
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
    elif event.type == pygame.MOUSEBUTTONDOWN:
        print("Botão do mouse pressionado!")
        print(event)
    elif event.type == pygame.MOUSEBUTTONUP:
        print("Botão do mouse liberado!")
        print(event)
```

Qual botão
foi
pressionado

A posição X,Y do
mouse no terminal

```
<Event(1026-MouseButtonUp {'pos': (428, 160), 'button': 1, 'touch': False, 'window': None})>
Botão do mouse pressionado!
<Event(1025-MouseButtonDown {'pos': (428, 160), 'button': 3, 'touch': False, 'window': None})>
Botão do mouse liberado!
```

Fonte: Elaborado pelo
autor (2026).

Pygame

36

- Desafio construa a interface **piano_final.py** – **Figura 26:**

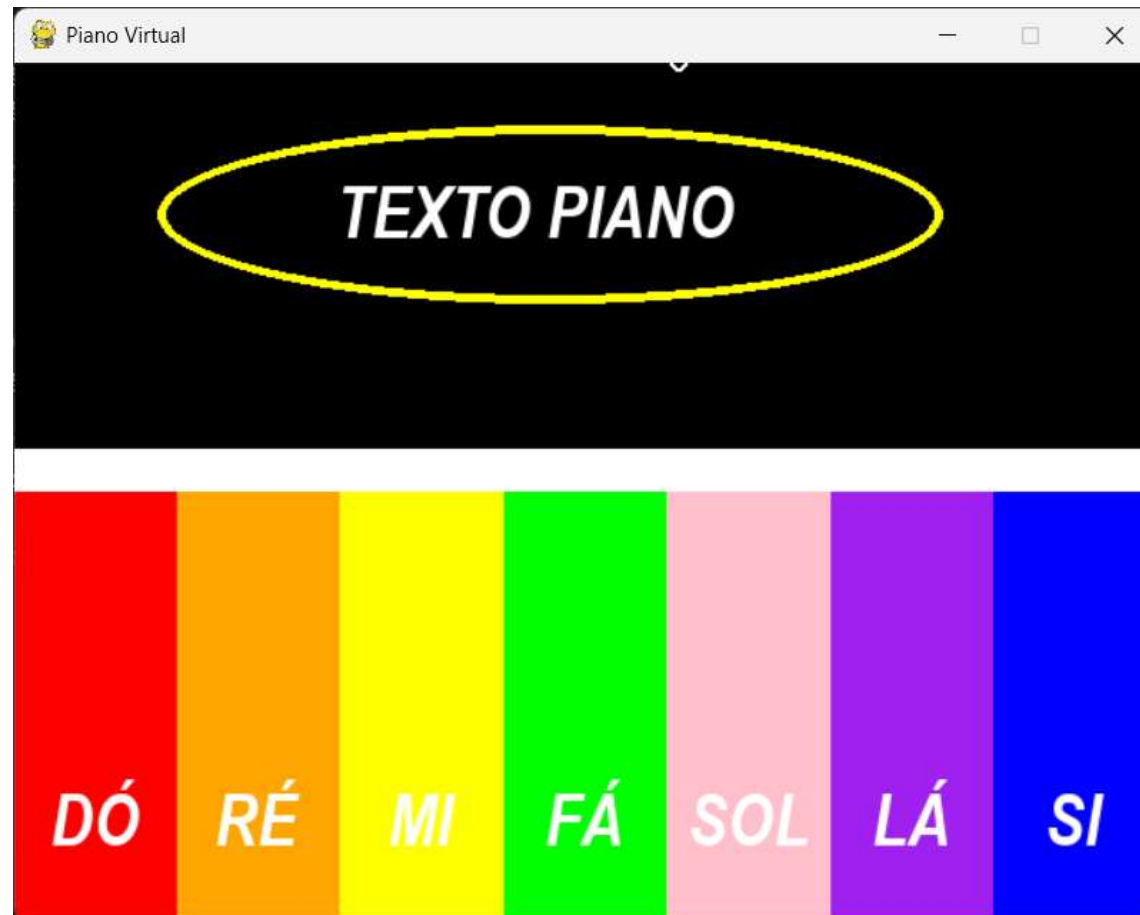


Figura 26 – PyGame
Piano Final

Fonte: Elaborado pelo
autor (2026).

Pygame

37

- **Cenário estático:** Basicamente é um jogo por sobreposição de imagens estáticas, sem a necessidade de prover a sensação de movimento dos elementos do jogo – Figura 27.

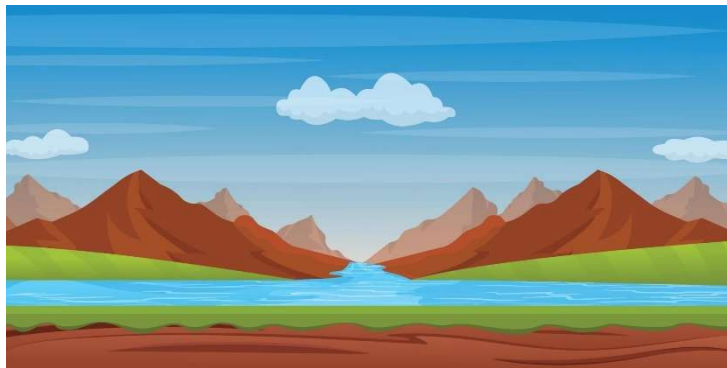
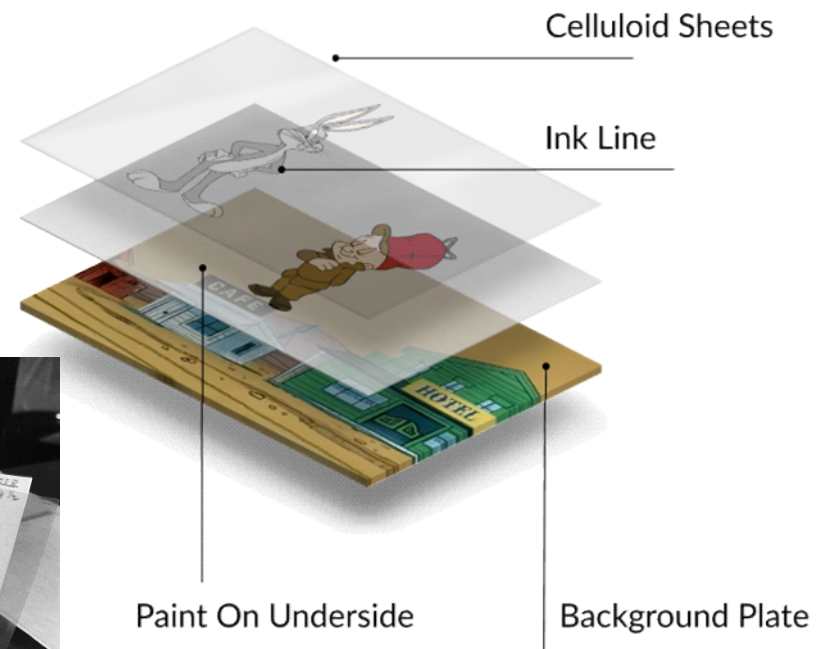


Figura 27 – Cenários Estáticos



Fonte: Google Images
(2026).

Pygame

38

- **Ícone:** Trabalhando com imagens 1/3. Crie o seguinte código, para o arquivo **icone.py** – **Figura 28.**

```
import pygame

pygame.init()
# Constantes para a tela
LARGURA_TELA = 640
ALTURA_TELA = 480
tela = pygame.display.set_mode((LARGURA_TELA, ALTURA_TELA))
# Título da janela
pygame.display.set_caption("Ícone da Janela")

clock = pygame.time.Clock()
running = True
# Ícone da janela
imagem = pygame.image.load("imagens/verde.png")
pygame.display.set_icon(imagem)
```

Fonte: Elaborado
pelo autor (2026).

Pygame

39

▣ Ícone: Trabalhando com imagens 2/3.

```
# Inicia o loop principal do jogo
while running:
    # Nesse ponto dentro desse laço é colocado o jogo para rodar,
    # ou seja, o que deve acontecer a cada frame

    # Observa os eventos de teclado e mouse
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Limpa a tela com uma cor de fundo antes de desenhar
    tela.fill("black")

    # Atualiza a tela com tudo o que foi desenhado acima
    pygame.display.flip()

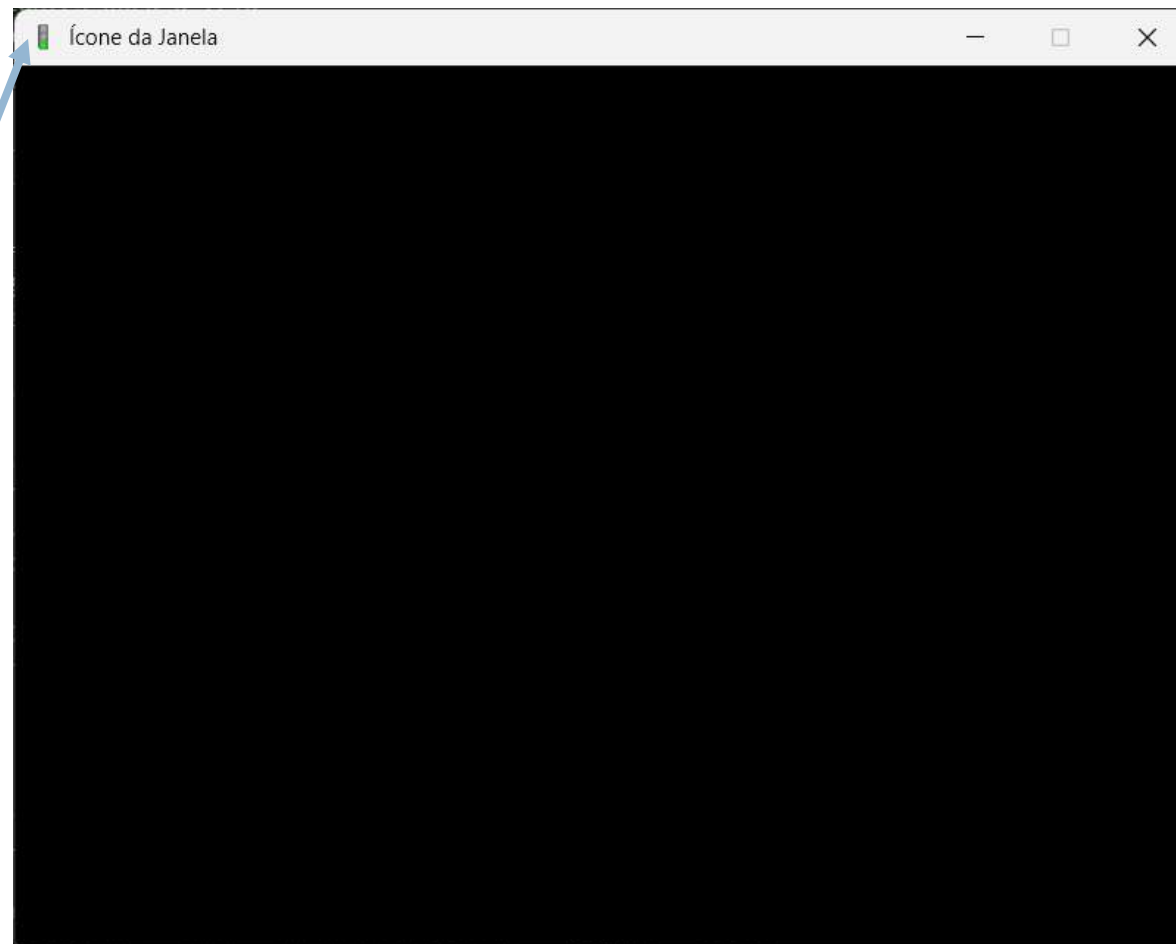
    # Controla a taxa de quadros (FPS)
    clock.tick(60)

pygame.quit()
```

Pygame

40

- ▣ **Ícone:** Trabalhando com imagens 3/3. Resultado:



Ícone
trocado.

Pygame

41

- ▣ **Sinaleiro:** Trabalhando com mais imagens 1/4. Crie o seguinte código, para o arquivo **sinaleiro.py** – **Figura 29**.

```
import time
import pygame

pygame.init()
# Constantes para a tela
LARGURA_TELA = 640
ALTURA_TELA = 480
tela = pygame.display.set_mode((LARGURA_TELA, ALTURA_TELA))
# Título da janela
pygame.display.set_caption("Sinaleiro")

clock = pygame.time.Clock()
running = True

verde = pygame.image.load("imagens/verde.png")
amarelo = pygame.image.load("imagens/amarelo.png")
vermelho = pygame.image.load("imagens/vermelho.png")
```

Fonte: Elaborado
pelo autor (2026).

Pygame

42

▣ **Sinaleiro:** Trabalhando com mais imagens 2/4.

```
# Inicia o loop principal do jogo
while running:
    # Nesse ponto dentro desse laço é colocado o jogo para rodar,
    # ou seja, o que deve acontecer a cada frame

    # Observa os eventos de teclado e mouse
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Limpa a tela com uma cor de fundo antes de desenhar
    tela.fill("black")

    tela.blit(verde, (0, 0))
    pygame.display.update()
    time.sleep(1)
```

Pygame

43

▣ **Sinaleiro:** Trabalhando com mais imagens 3/4.

```
tela.blit(amarelo, (0, 0))  
pygame.display.update()  
time.sleep(1)
```

```
tela.blit(vermelho, (0, 0))  
pygame.display.update()  
time.sleep(1)
```

```
# Atualiza a tela com tudo o que foi desenhado acima  
pygame.display.flip()
```

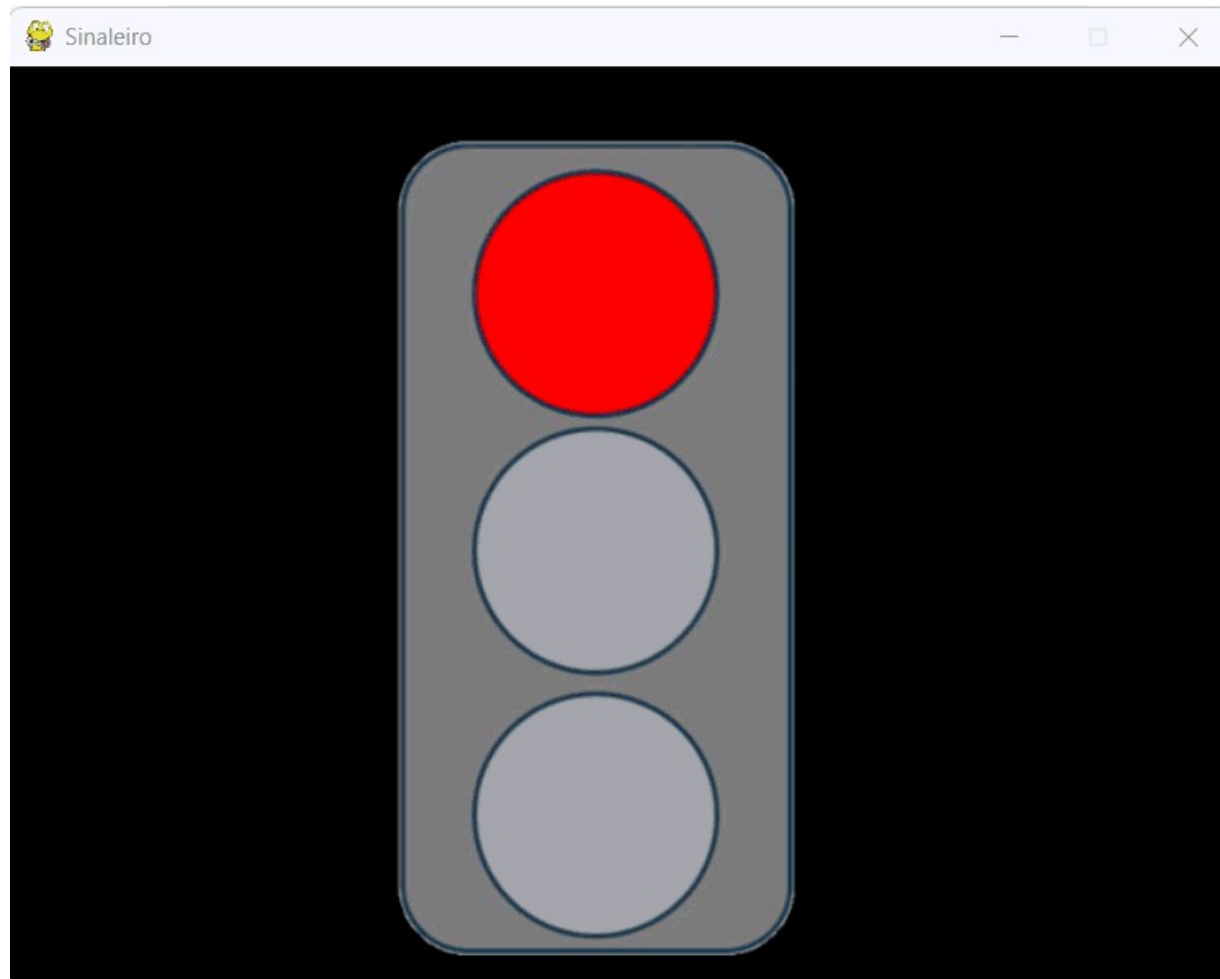
```
# Controla a taxa de quadros (FPS)  
clock.tick(60)
```

```
pygame.quit()
```

Pygame

44

- ▣ **Sinaleiro:** Trabalhando com mais imagens 4/4. Resultado:



OpenCV e NumPy

45

- **OpenCV:** O **OpenCV** (*Open Source Computer Vision Library*) é a biblioteca de código aberto mais popular do mundo voltada para **Visão Computacional** e **Processamento de Imagens**. Lançada originalmente pela Intel em 2000, ela é multiplataforma (roda em Windows, Linux, macOS, Android e iOS) e possui suporte para diversas linguagens de programação, sendo **Python**, **C++** e **Java** as principais suportadas.
- **Versão atual:** 5.0.0;
- **Lançamento:** 06/2026;
- **Compatível:** Linux, Mac e Windows.

OpenCV e NumPy

46

- ▣ **NumPy:** O NumPy (*Numerical Python*) é a biblioteca mais fundamental para computação científica e análise de dados no ecossistema Python. Ela é a base sobre a qual praticamente todas as outras ferramentas famosas de Ciência de Dados e Machine Learning (como Pandas, Scikit-Learn, SciPy e o próprio OpenCV) são construídas.
- ▣ **Versão atual:** 2.3.0;
- ▣ **Lançamento:** 05/2026. Figura 30

Figura 30 – Logos
OpenCV e NumPy



Fonte: Google Images
(2026).

OpenCV e NumPy

47

▣ Ambiente Virtual:

- `python -m venv libcurso`
- `libcurso\Scripts\activate`
- **Obs:** Em caso de erro:
 - `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process`
 - `libcurso\Scripts\activate`
 - **O terminal será assim agora: (libcurso) PS C:\dev\testeenv>**

▣ OpenCV: Instalar via terminal:

- `pip install opencv-python`
- **Ou:** `pip install opencv-python --user`

▣ NumPy: Instalar via terminal:

- `pip install numpy`
- **Ou:** `pip install numpy --user`

OpenCV e NumPy

48

▣ Transformação de Perspectiva – Figura 31

Figura 31 – Transformação de Perspectiva



Fonte: Elaborado pelo autor (2026).

OpenCV e NumPy

49

- **Funções do OpenCV:** As funções nesta seção realizam diversas transformações geométricas de imagens 2D. Eles não alteram o conteúdo da imagem, mas deformam a grade de pixels e mapeiam essa grade deformada para a imagem de destino.
 - **getPerspectiveTransform:** Calcula uma transformação de perspectiva a partir de quatro pares de pontos correspondentes.
 - **warpPerspective:** Aplica uma transformação de perspectiva a uma imagem.

Fonte:

https://docs.opencv.org/5.0/main_modules/geometry_shape.html#getperspectivetransform

OpenCV e NumPy

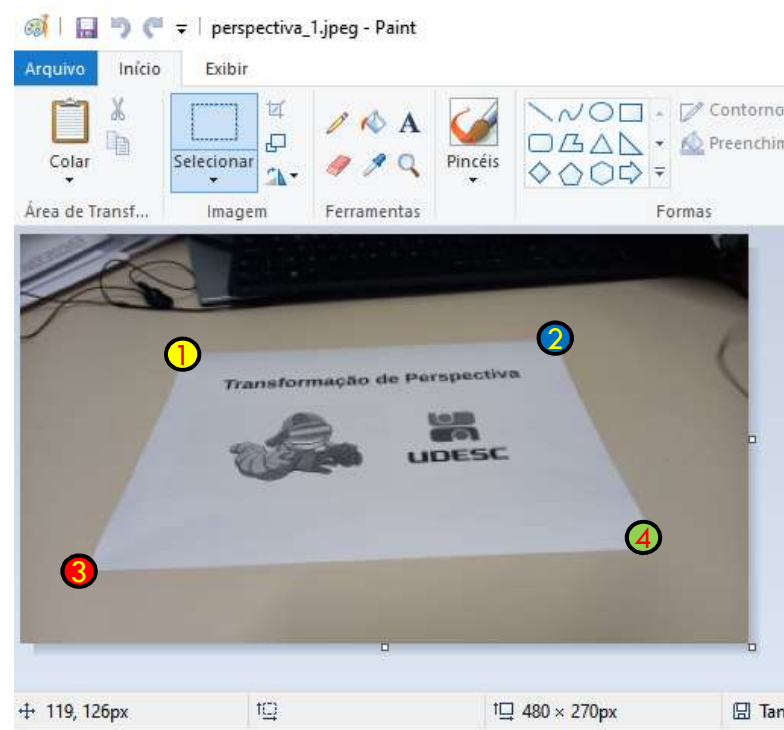
50

- **Transformação de Perspectiva de forma “Manual”:** Abra a figura a seguir no MS Paint e descubra a posição x,y dos 4 pontos – Figura 32.

Figura 32 – Transformação de
(0,0) Perspectiva Paint



(480,270)



Fonte: Elaborado pelo autor (2026).

OpenCV e NumPy

51

- A marcação tende a ter essa saída de coordenadas a depender aonde foi clicado. Figura 33

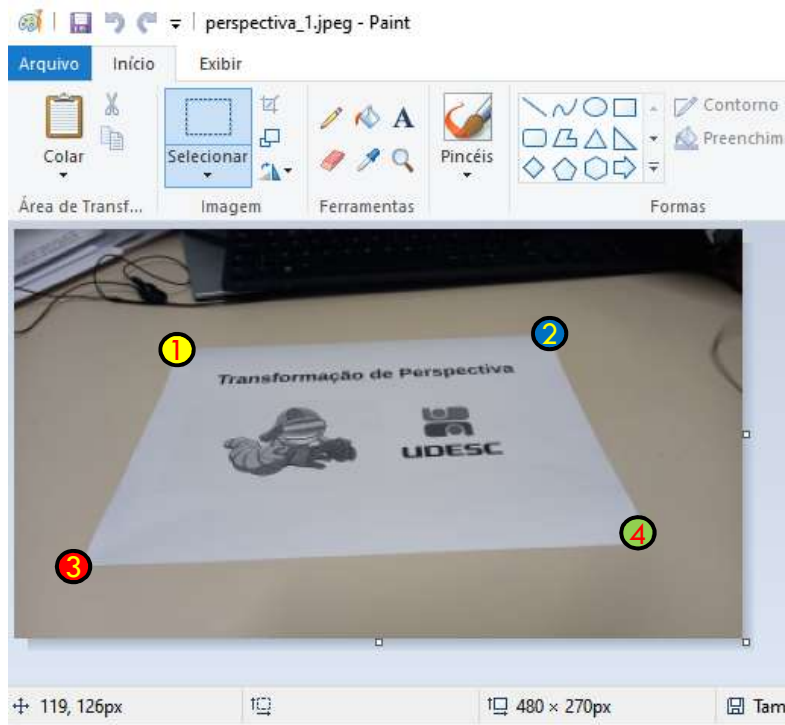


Figura 33 – Transformação de Perspectiva Paint e Cores

- ① [110, 78]
- ② [352, 70]
- ③ [44, 222]
- ④ [420, 208]

OpenCV e NumPy

52

- ▣ Codificando a transformação manual com base nas coordenadas do **MS Paint – 1/4, perspectiva_manual.py – Figura 34.**

```
import cv2
import numpy as np

# Imagem original e a tela final que será projetada a transformação
imagem = cv2.imread("imagens/perspectiva_original.jpeg")
LARGURA, ALTURA = 480, 270

pontos_1 = np.float32([[110, 78], [352, 70], [44, 222], [420, 208]])
pontos_2 = np.float32([[0, 0], [LARGURA, 0], [0, ALTURA], [LARGURA, ALTURA]])
# Transformação de perspectiva
matrix = cv2.getPerspectiveTransform(pontos_1, pontos_2)
imagem_corrigida = cv2.warpPerspective(imagem, matrix, (LARGURA, ALTURA))

# Lista de cores para cada círculo (RGB)
cores = [(0, 255, 255), (255, 0, 0), (0, 0, 255), (0, 255, 0)]
```

Figura 34 – NumPy.

OpenCV e NumPy

53

- ▣ Codificando a transformação manual com base nas coordenadas do **MS Paint – 2/4.**

```
# Loop para desenhar os círculos e os números
for i, ponto in enumerate(pontos_1):
    centro_x = int(ponto[0])
    centro_y = int(ponto[1])

    # Desenha o círculo nos vértices
    cv2.circle(imagem, (centro_x, centro_y), 8, cores[i], cv2.FILLED)

    # Configurações do texto/número
    texto = str(i + 1) # Número que será exibido (1, 2, 3, 4)
    fonte = cv2.FONT_HERSHEY_SIMPLEX
    escala = 0.4
    cor_texto = (0, 0, 0) # Preto para dar contraste
    espessura = 1

    # Descobre o tamanho do texto para centralizar
    (t_largura, t_altura), _ = cv2.getTextSize(texto, fonte, escala, espessura)
```

OpenCV e NumPy

54

- Codificando a transformação manual com base nas coordenadas do **MS Paint – 3/4**.

```
# Calcula a posição para o texto ficar no centro do círculo
texto_x = centro_x - t_largura // 2
texto_y = centro_y + t_altura // 2

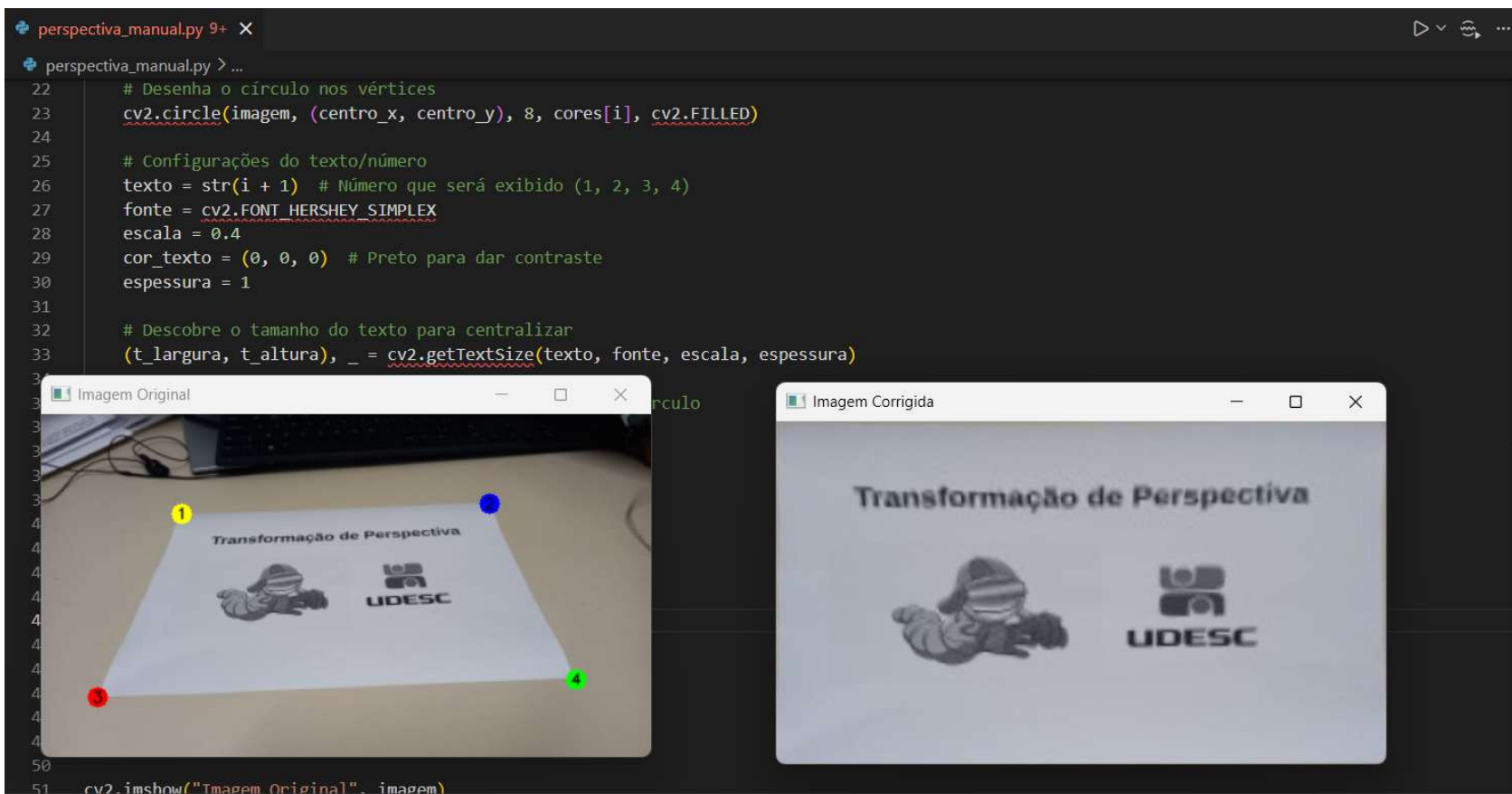
# Desenha o número
cv2.putText(
    imagem,
    texto,
    (texto_x, texto_y),
    fonte,
    escala,
    cor_texto,
    espessura,
    cv2.LINE_AA,
)

cv2.imshow("Imagem Original", imagem)
cv2.imshow("Imagem Corrigida", imagem_corrigida)
cv2.waitKey(0)
cv2.destroyAllWindows()
```


OpenCV e NumPy

55

- Codificando a transformação manual com base nas coordenadas do **MS Paint** – 4/4. Resultado:



OpenCV e NumPy

56

- Transformação usando o mouse 1/7,
perspectiva_mouse.py. – Figura 35

```
import cv2
import numpy as np
import pygame

# Inicialização do Pygame e do módulo de fontes
pygame.init()
pygame.font.init()

# Constantes para a tela
LARGURA, ALTURA = 480, 270
tela = pygame.display.set_mode((LARGURA, ALTURA))
pygame.display.set_caption("Seleção de Pontos para Perspectiva")

# Configuração da Fonte
fonte = pygame.font.SysFont("Arial", 14, bold=True)

# Carregamento de recursos
CAMINHO_IMAGEM = "imagens/perspectiva_original.jpeg"
imagem_original = cv2.imread(CAMINHO_IMAGEM)
imagem_pygame = pygame.image.load(CAMINHO_IMAGEM)
```

Figura 35 – PyGame e OpenCV.

Fonte: Elaborado pelo autor (2026).

OpenCV e NumPy

57

▣ Transformação usando o **mouse 2/7**.

```
# Estado da aplicação
vertices = np.zeros((4, 2), dtype=np.float32)
contador = 0
transformacao_realizada = False

# Lista de cores para cada círculo (RGB)
cores = [
    (255, 255, 0), # 1º Ponto: Amarelo
    (0, 0, 255), # 2º Ponto: Azul
    (255, 0, 0), # 3º Ponto: Vermelho
    (0, 255, 0), # 4º Ponto: Verde
]

clock = pygame.time.Clock()
running = True
```

OpenCV e NumPy

58

■ Transformação usando o **mouse 3/7**.

```
def aplicar_perspectiva(pontos_origem: np.ndarray) -> None:
    """Calcula e exibe a transformação de perspectiva."""
    # Mapeamento do destino baseado na ordem esperada dos cliques
    pontos_destino = np.float32(
        [
            [0, 0], # Superior Esquerdo
            [LARGURA, 0], # Superior Direito
            [0, ALTURA], # Inferior Esquerdo
            [LARGURA, ALTURA], # Inferior Direito
        ]
    )

    # Gera a matriz de transformação
    matrix = cv2.getPerspectiveTransform(pontos_origem, pontos_destino)

    # Aplica o warp na imagem original do OpenCV
    imagem_corrigida = cv2.warpPerspective(
        imagem_original, matrix, (LARGURA, ALTURA)
    )

    # Abre a janela nativa do OpenCV com o resultado
    cv2.imshow("Imagem Corrigida", imagem_corrigida)
```

OpenCV e NumPy

59

▣ Transformação usando o **mouse 4/7**.

```
# Laço Principal
while running:

    # Observa os eventos de teclado e mouse
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        elif event.type == pygame.MOUSEBUTTONDOWN and contador < 4:
            # Captura a posição exata do clique
            vertices[contador] = event.pos
            contador += 1

    # Limpa a tela com uma cor de fundo antes de desenhar
    tela.fill("black")
    tela.blit(imagem_pygame, (0, 0))
```

OpenCV e NumPy

60

- Transformação usando o mouse 5/7, **perspectiva_mouse.py.**

```
# Desenha os círculos e os números centralizados dentro deles
for i in range(contador):
    x = int(vertices[i][0])
    y = int(vertices[i][1])

    # Desenha o círculo (raio 8 para dar espaço ao número)
    pygame.draw.circle(tela, cores[i], (x, y), 8)

    # Renderiza o número em texto preto para dar contraste com as cores de fundo
    texto_superficie = fonte.render(str(i + 1), True, (0, 0, 0))

    # Alinha o centro do retângulo do texto com o centro do clique
    texto_retangulo = texto_superficie.get_rect(center=(x, y))

    # Desenha o texto na tela
    tela.blit(texto_superficie, texto_retangulo)
```

OpenCV e NumPy

61

- ▣ Transformação usando o **mouse 6/7**.

```
# Processa a perspectiva uma única vez após capturar os 4 pontos
if contador == 4 and not transformacao_realizada:
    aplicar_perspectiva(vertices)
    transformacao_realizada = True

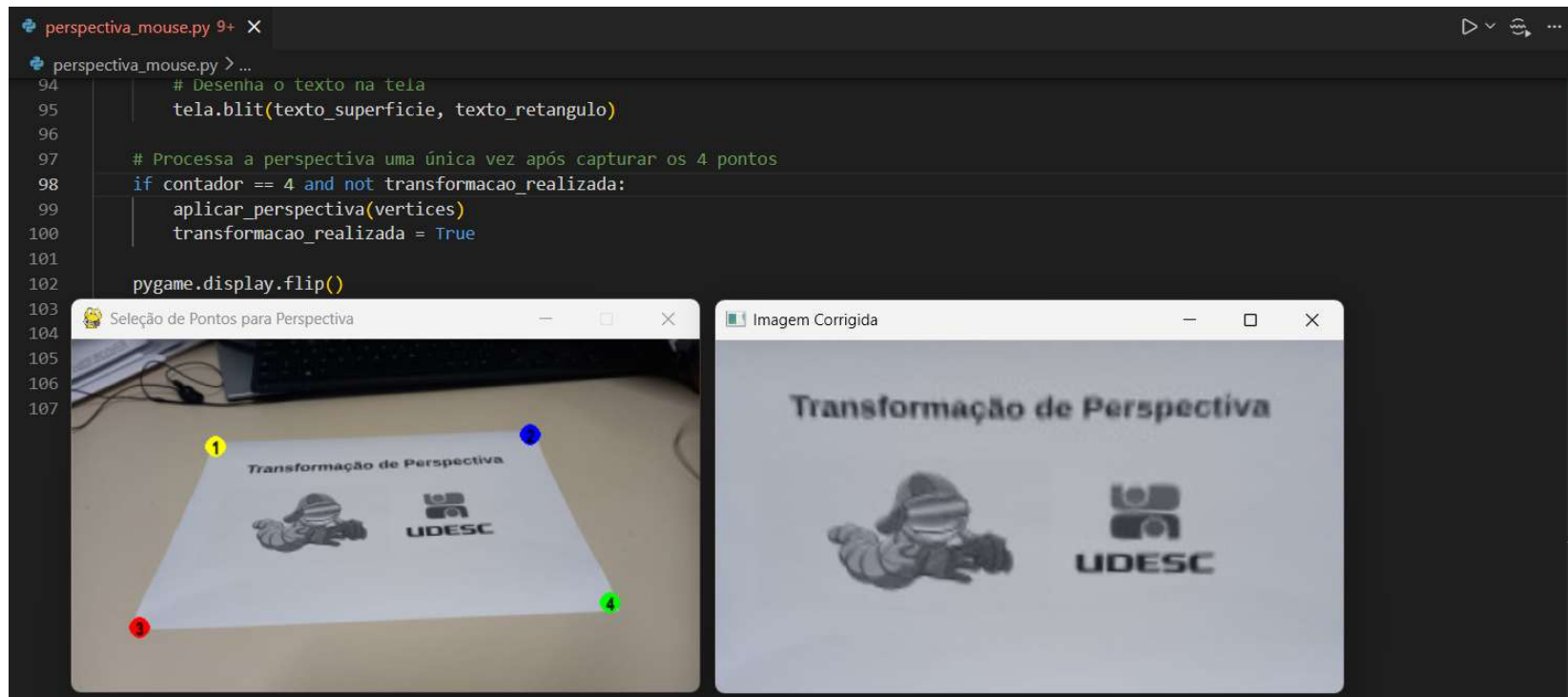
pygame.display.flip()
clock.tick(60)

pygame.quit()
cv2.destroyAllWindows()
```

OpenCV e NumPy

62

- Transformação usando o **mouse 7/7**.



MediaPipe

63

- ▣ **MediaPipe:** O **MediaPipe** é um framework de código aberto desenvolvido pela Google, projetado para facilitar a criação e a implementação de pipelines de aprendizado de máquina (***Machine Learning***) voltados para o processamento de mídias estruturadas, como fluxos de vídeo ao vivo, áudio e imagens. Ele foi pensado para rodar de maneira extremamente rápida e leve diretamente em dispositivos finais (***on-device***), como smartphones, computadores comuns e navegadores web.
- ▣ **Versão atual:** 0.10.35;
- ▣ **Lançamento:** 04/2026.

MediaPipe

64

▣ Site

—

Figura

36:

<https://developers.google.com/edge/mediapipe/solutions/guide>

Figura 36 – Google
MediaPipe



Solution	Android	Web	Python	iOS	Customize model
LLM Inference API	●	●		●	●
Object detection	●	●	●	●	●
Image classification	●	●	●	●	●
Image segmentation	●	●	●		
Interactive segmentation	●	●	●		
Hand landmark detection	●	●	●	●	
Gesture recognition	●	●	●	●	●
Image embedding	●	●	●		
Face detection	●	●	●	●	
Face landmark detection	●	●	●		
Pose landmark detection	●	●	●		
Image generation	●				●
Text classification	●	●	●	●	●
Text embedding	●	●	●		
Language detector	●	●	●		
Audio classification	●	●	●		

Fonte: Elaborado pelo autor (2026).

MediaPipe

65

□ Posição

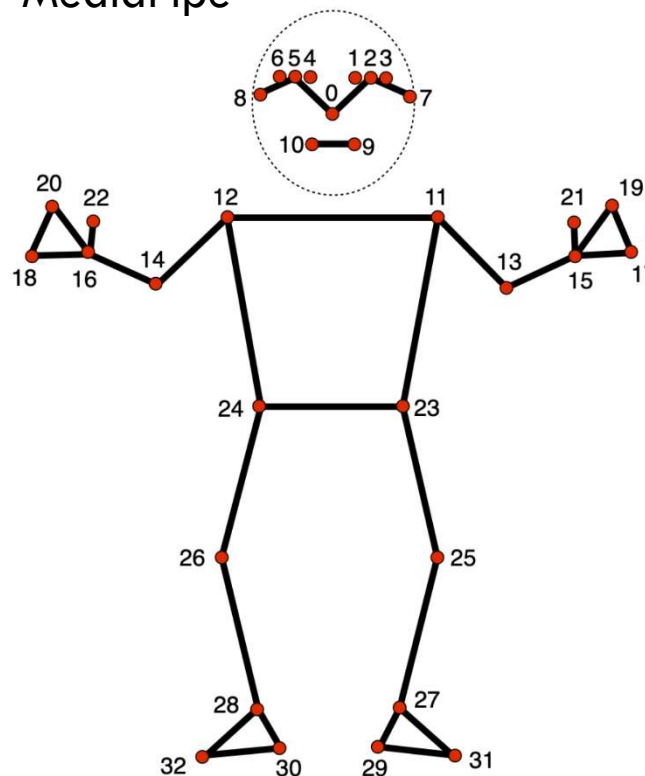
https://developers.google.com/edge/mediapipe/solutions/vision/pose_landmarker

Figura

37:

Figura 37 – Posições
MediaPipe

0 - nose
1 - left eye (inner)
2 - left eye
3 - left eye (outer)
4 - right eye (inner)
5 - right eye
6 - right eye (outer)
7 - left ear
8 - right ear
9 - mouth (left)
10 - mouth (right)
11 - left shoulder
12 - right shoulder
13 - left elbow
14 - right elbow
15 - left wrist
16 - right wrist



17 - left pinky
18 - right pinky
19 - left index
20 - right index
21 - left thumb
22 - right thumb
23 - left hip
24 - right hip
25 - left knee
26 - right knee
27 - left ankle
28 - right ankle
29 - left heel
30 - right heel
31 - left foot index
32 - right foot index

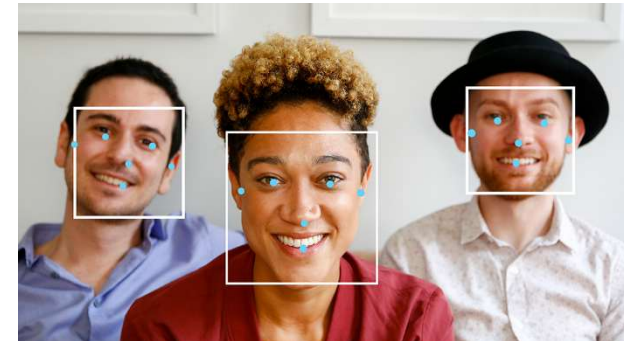
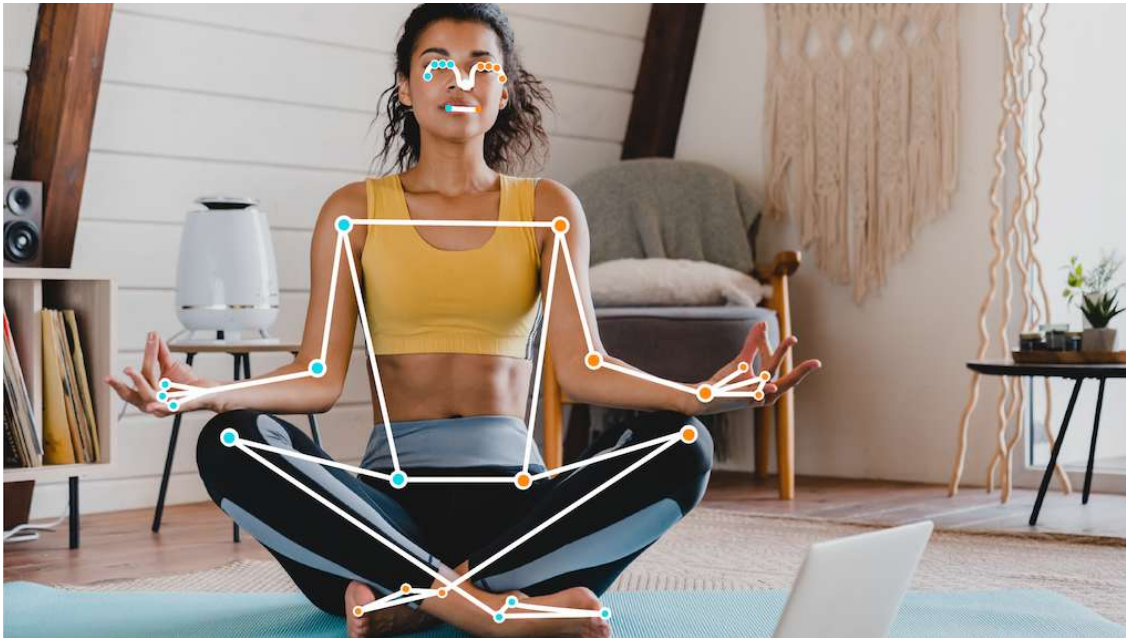
Fonte: Elaborado pelo autor (2026).

MediaPipe

66

▣ Exemplos – Figura 38:

Figura 38 – Exemplos MediaPipe



Fonte: Elaborado pelo autor (2026).

MediaPipe

67

- **Atualmente:** O Google mudou reformulou o ecossistema substituindo as antigas soluções (**Legacy Solutions**) pelo novo modelo chamado **MediaPipe Tasks** – **Figura 39**.
- Baixar os modelos:
 - https://developers.google.com/edge/mediapipe/solutions/vision/pose_landmarker

Figura 39 – MediaPipe Tasks

Model bundle	Input shape	Data type	Model Cards	Versions
Pose landmark (lite)	Pose detector: 224 x 224 x 3 Pose landmarker: 256 x 256 x 3	float 16	info	Latest
Pose landmark (Full)	Pose detector: 224 x 224 x 3 Pose landmarker: 256 x 256 x 3	float 16	info	Latest
Pose landmark (Heavy)	Pose detector: 224 x 224 x 3 Pose landmarker: 256 x 256 x 3	float 16	info	Latest

Fonte: Elaborado pelo autor (2026).

Juntando as partes

68

▣ MediaPipe 1/3, mediapipevideo.py – Figura 40:

```
import cv2
import mediapipe as mp
from mediapipe.framework.formats import landmark_pb2

# Configuração do PoseLandmarker para rodar em modo de imagem
options = mp.tasks.vision.PoseLandmarkerOptions(
    base_options=mp.tasks.BaseOptions(
        model_asset_path="pose_landmarker_full.task"
    ),
    running_mode=mp.tasks.vision.RunningMode.IMAGE,
    min_pose_detection_confidence=0.5,
    min_pose_presence_confidence=0.5,
    min_tracking_confidence=0.5,
)

cap = cv2.VideoCapture("video/video_para_testar_mediapipe.mp4")

with mp.tasks.vision.PoseLandmarker.create_from_options(options) as landmarker:
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        annotated_image = frame.copy()

        # Converte o frame e roda o detector
        mp_image = mp.Image(
            image_format=mp.ImageFormat.SRGB,
            data=cv2.cvtColor(frame, cv2.COLOR_BGR2RGB),
        )
```

Figura 40 – MediaPipe
Camera

Fonte: Elaborado pelo autor (2026).

Juntando as partes

69

▣ MediaPipe 2/3.

```
results = landmarker.detect(mp_image)

if results.pose_landmarks:
    for pose_landmarks in results.pose_landmarks:
        # Converte os landmarks para o formato do desenho
        proto = landmark_pb2.NormalizedLandmarkList()
        proto.landmark.extend(
            [
                landmark_pb2.NormalizedLandmark(x=l.x, y=l.y, z=l.z)
                for l in pose_landmarks
            ]
        )

        # Desenha apenas na imagem anotada
        mp.solutions.drawing_utils.draw_landmarks(
            annotated_image, proto, mp.solutions.pose.POSE_CONNECTIONS
        )

        # Print do Nariz (ID 0)
        nose = pose_landmarks[0]
        print(f"Nariz -> X: {nose.x:.4f}, Y: {nose.y:.4f}")

        # Agora temos as duas telas de volta!
        cv2.imshow("MediaPipe Tasks", annotated_image)
        cv2.imshow("Original", frame)

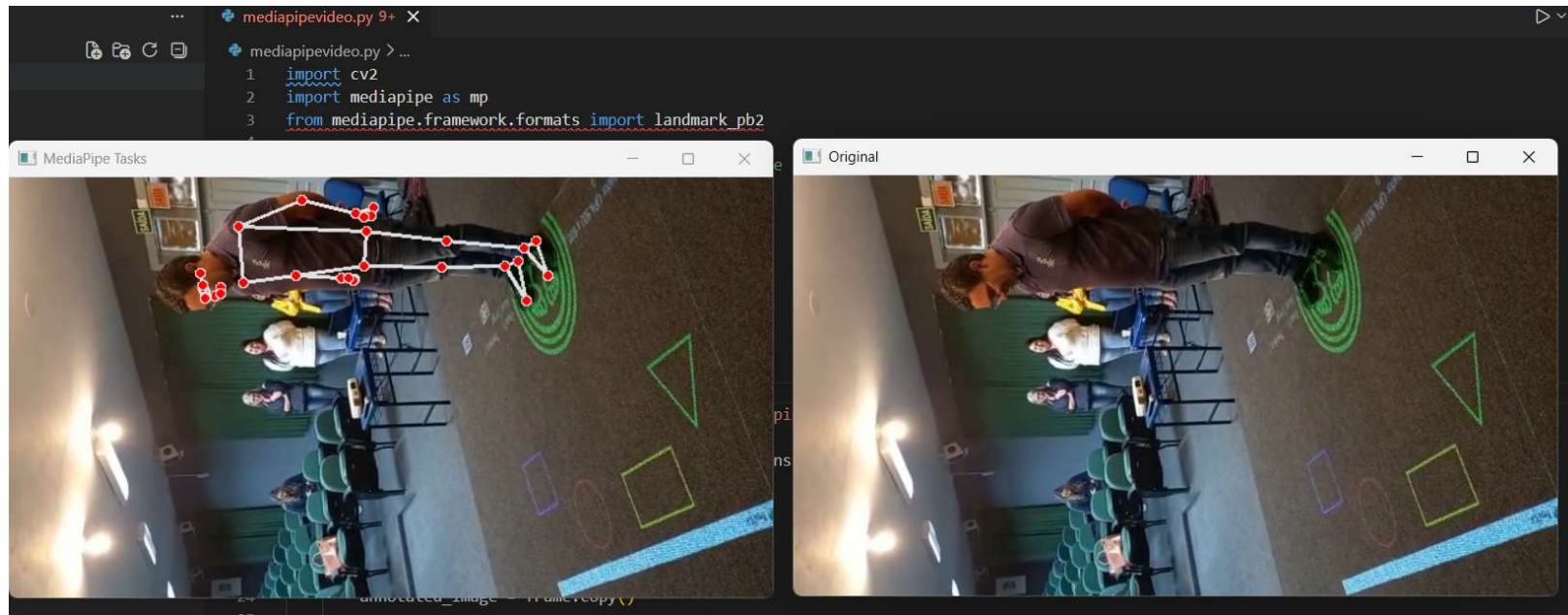
        if cv2.waitKey(10) & 0xFF == ord("q"):
            break

cap.release()
cv2.destroyAllWindows()
```


Juntando as partes

70

MediaPipe 3/3.



Juntando as partes

71

- ▣ MediaPipe, Opencv e PyGame, **mediapipeall.py**. Testar com o fonte disponível no git.

E agora?

72

■ E agora?

- Um bom projeto começa com a organização das suas pastas;
- Testes as possibilidade de detecção com o tasks do MediaPipe;
- Faça pequenos trechos de código pygame e aos poucos envolva o Opencv e MediaPipe.

E agora?

73

■ Questionário:



Referências

74

- GOOGLE. **MediaPipe Solutions guide.** Disponível em: <https://developers.google.com/edge/mediapipe/solutions/guide>. Acesso em: 18 jun. 2026.
- GOOGLE Images. 2026. Disponível em: <https://images.google.com/>. Acesso em: 18 jun. 2026.
- MICROSOFT. **The open source AI code editor.** Disponível em: <https://code.visualstudio.com/>. Acesso em: 18 jun. 2026.
- NUMPY. NumPy. Disponível em: <https://numpy.org/>. Acesso em: 18 jun. 2026.
- OPENCV. **OpenCV - Open Computer Vision Library.** Disponível em: <https://opencv.org/>. Acesso em: 18 jun. 2026.
- PYGAME. 2026. Disponível em: <https://www.pygame.org/news>. Acesso em: 18 jun. 2026.
- PYGAME CE. 2026. Disponível em: <https://pyga.me/>. Acesso em: 18 jun. 2026.
- PYTHON. 2026. Disponível em: <https://www.python.org/downloads/windows>. Acesso em: 18 jun. 2026.
- THE Python Arcade Library. 2026. Disponível em: <https://api.arcade.academy/en/stable>. Acesso em: 18 jun. 2026.
- TRINDADE, André Bonetto; PEREIRA, Gabriel Brunelli; DA SILVA HOUNSELL, Marcelo. Chão Interativo e Jogos Sérios Ativos para Autistas: A Plataforma T-TEA e o Jogo RepeTEA. In: **Simpósio Brasileiro de Jogos e Entretenimento Digital (SBGames)**. SBC, 2022. p. 512-521.